Oregon State University

School of EECS

ECE 499/599 – Digital VLSI Design Automation

Laboratory Design Project Report

Name: Yu-Chuan Tseng

Student ID: 934619336

1. Introduction

This project implements a 4-bit serial-input, serial-output multiplier using CMOS standard cell design flow, including:

- Verilog design
- Simulation (Xcelium/SimVision)
- Synthesis (Genus)
- Place & Route (Innovus)

The system accepts serial inputs and produces serial outputs controlled by an FSM.

2. Core Functional Blocks

2.1 4-bit x 4-bit Multiplier

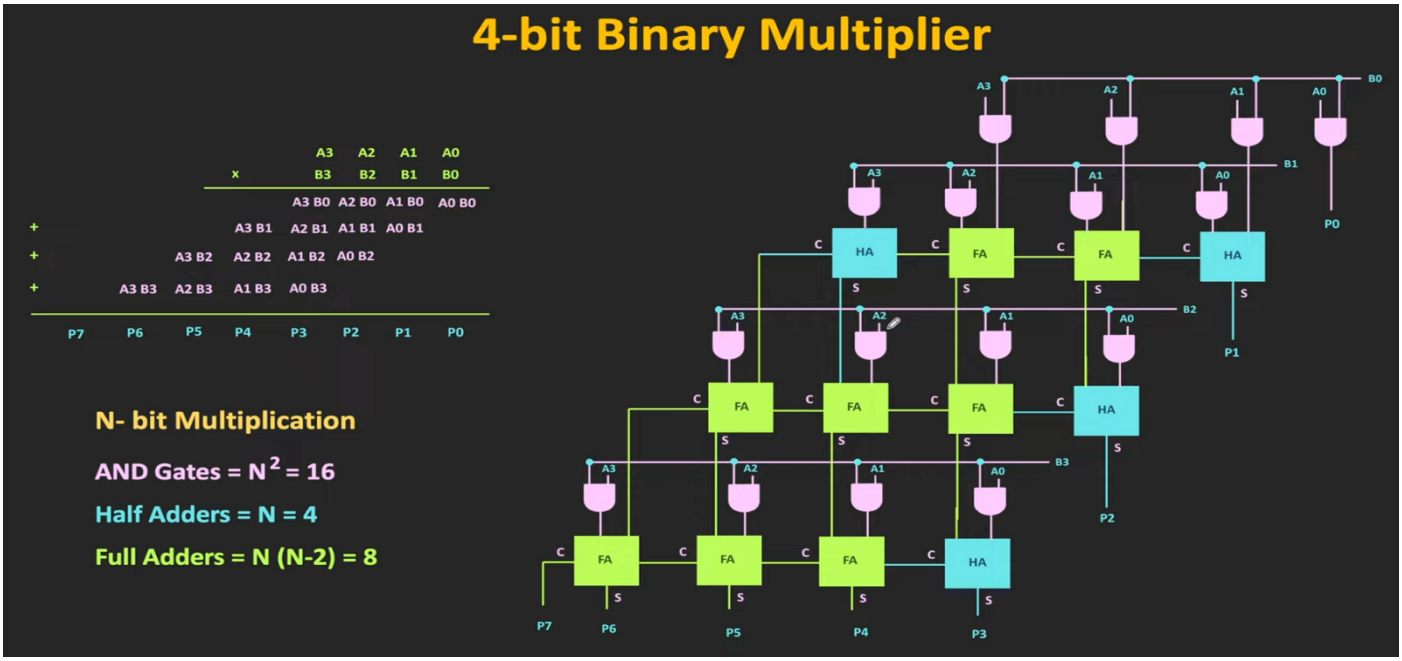
Functional Description

The 4-bit × 4-bit multiplier is a purely combinational circuit that generates an 8-bit output from two 4-bit inputs A and B.

Partial products are first generated using bitwise AND operations between bits of A and B. These partial products are then summed using half-adders and full-adders arranged in a column-wise structure.

The computation is completed within a single clock cycle, and the output remains stable as long as the inputs do not change.

Block Diagram



Verilog Implementation& Testbench

tsenyuch@eng-analogsim3:~/ECE499/cadence/final_project/Multiplier599

GNU nano 5.6.1 multiplier4x4.v

```
wire s6,  c6;

// ====================================================
// Output bit P[0]
// ====================================================
assign P[0] = p00;

// ====================================================
// Column for P[1]
// P[1] = p10 + p01
// ====================================================
ha HA1 (
    .a    (p10),
    .b    (p01),
    .sum  (P[1]),
    .carry(c1)
);

// ====================================================
// Column for P[2]
// bits involved: p20, p11, p02, and carry from previous col
// ====================================================
fa FA1 (
    .a    (p20),
    .b    (p11),
    .cin  (p02),
    .sum  (s2a),
    .carry(c2a)
);

ha HA2 (
    .a    (s2a),
    .b    (c1),
    .sum  (P[2]),
    .carry(c2b)
);
```

tsenyuch@eng

GNU nano 5.6.1

```
`timescale 1ns/1ps

module tb_multiplier4x4;

reg  [3:0] A;
reg  [3:0] B;
wire [7:0] P;

// Instantiate the multiplier
multiplier4x4 uut (
    .A(A),
    .B(B),
    .P(P)
);

initial begin

    $display("A  B  |  P (Expected)");
    $display("---------------------");

    A = 4'd3;  B = 4'd5;  #10;
    $display("%d  %d  |  %d (15)", A, B, P);

    A = 4'd7;  B = 4'd9;  #10;
    $display("%d  %d  |  %d (63)", A, B, P);

    A = 4'd15; B = 4'd15; #10;
    $display("%d  %d  |  %d (225)", A, B, P);

    A = 4'd4;  B = 4'd6;  #10;
    $display("%d  %d  |  %d (24)", A, B, P);

    A = 4'd0;  B = 4'd12; #10;
    $display("%d  %d  |  %d (0)", A, B, P);

    A = 4'd10; B = 4'd3;  #10;
    $display("%d  %d  |  %d (30)", A, B, P);

    $display("---------------------");
    $display("Simulation finished.");

    $finish;

end

endmodule
```

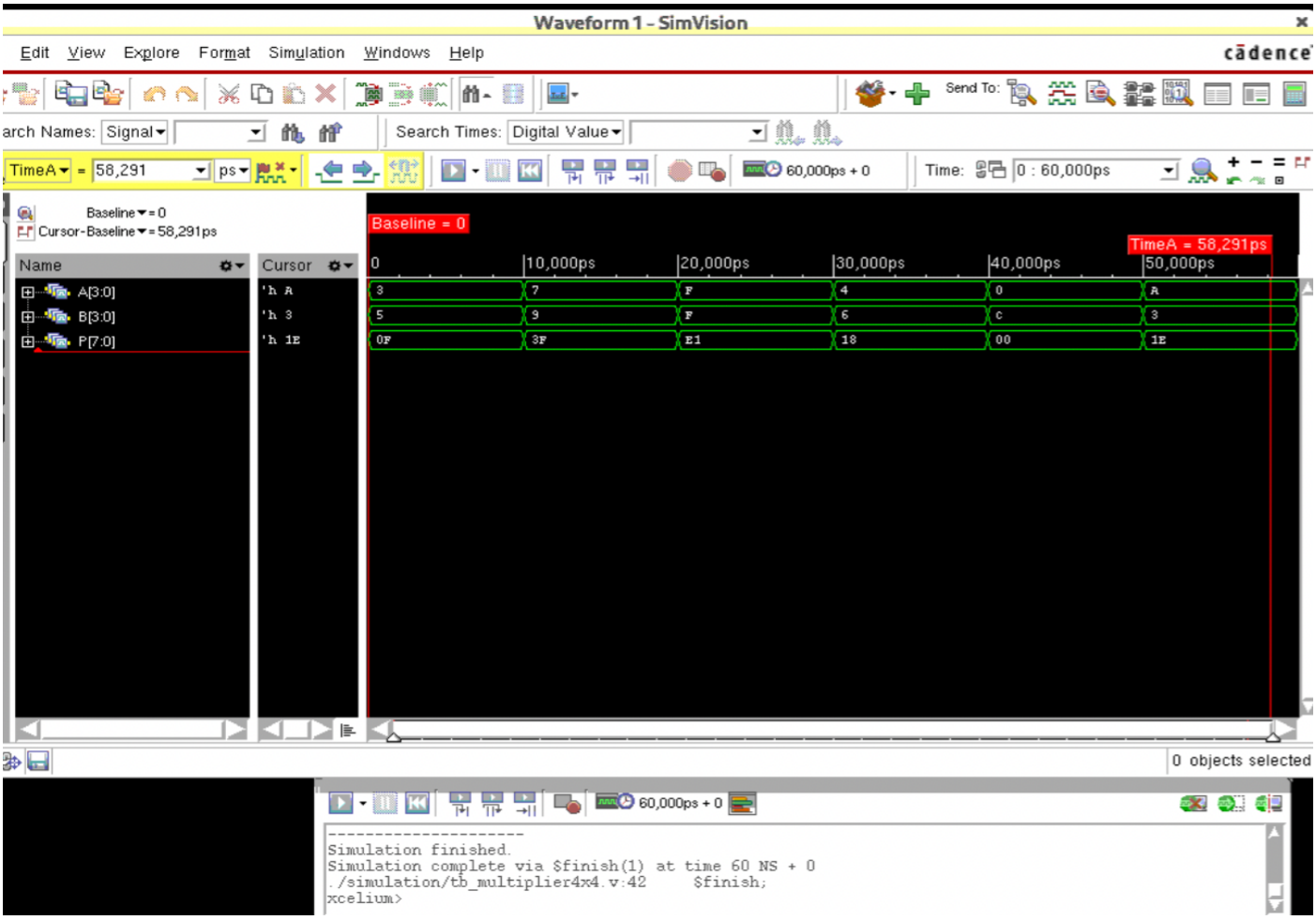
Simulation Results

The 4-bit multiplier is implemented using a structural design approach. Partial products are first generated using bitwise AND operations, and the final result is obtained through column-wise accumulation.

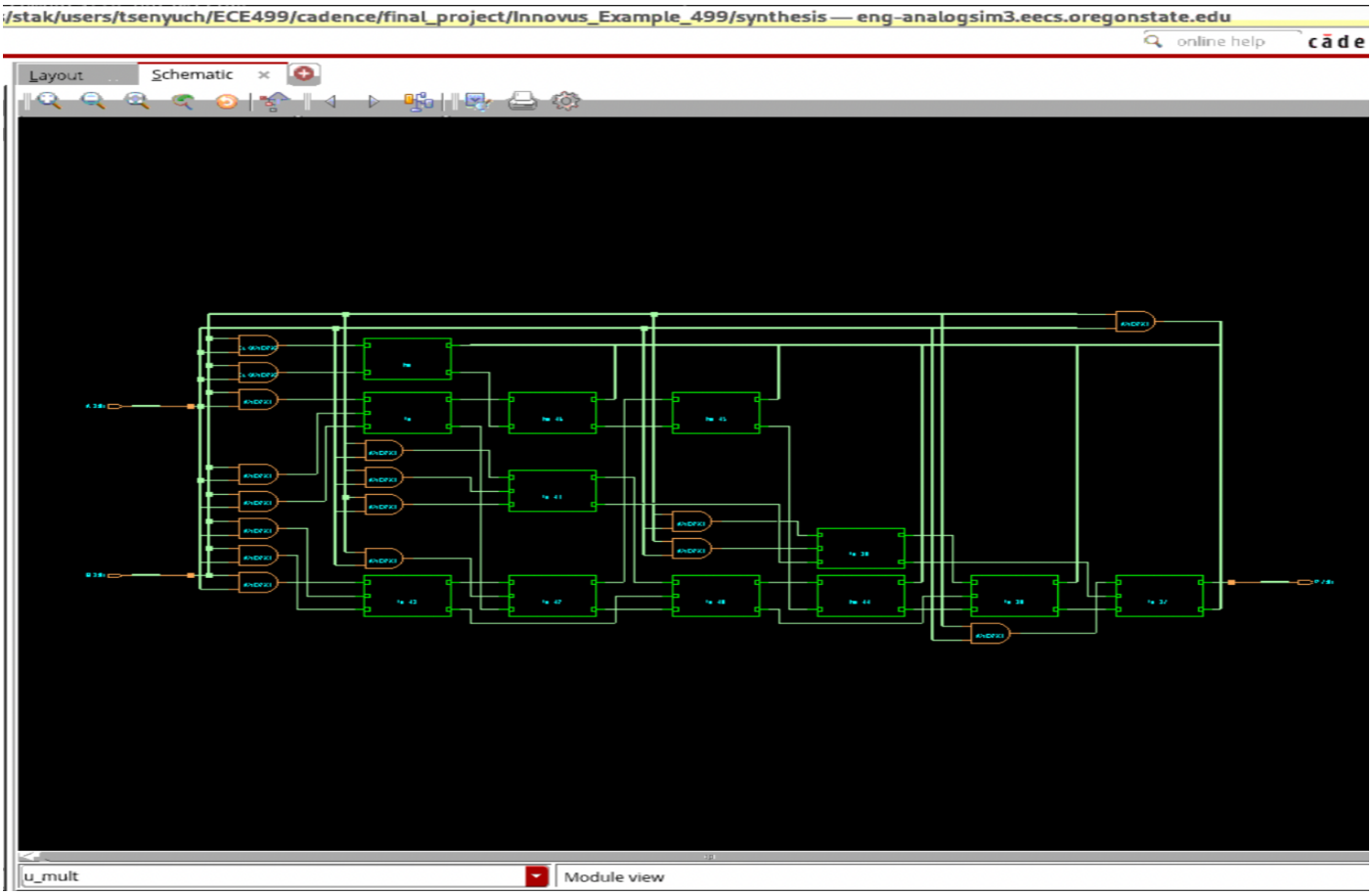
Each output bit is computed using a combination of half-adders (HA) and full-adders (FA). For example, the least significant bit P[0] is directly assigned from the partial product p00. The next bit P[1] is computed using a half-adder that sums p10 and p01.

For higher-order bits such as P[2], multiple partial products and carry signals are combined using a full-adder followed by a half-adder, demonstrating carry propagation across columns.

This structure ensures a purely combinational implementation and clearly exposes the critical path through cascaded adders.



Synthesis Results (Genus)



2.2 8-bit Serial-to-Parallel

Functional Description:

The S2P module converts serial input data into an 8-bit parallel output.

Inputs:

- clk: system clock

- reset: clears register

- shift_en: enables shifting

- in: serial input bit

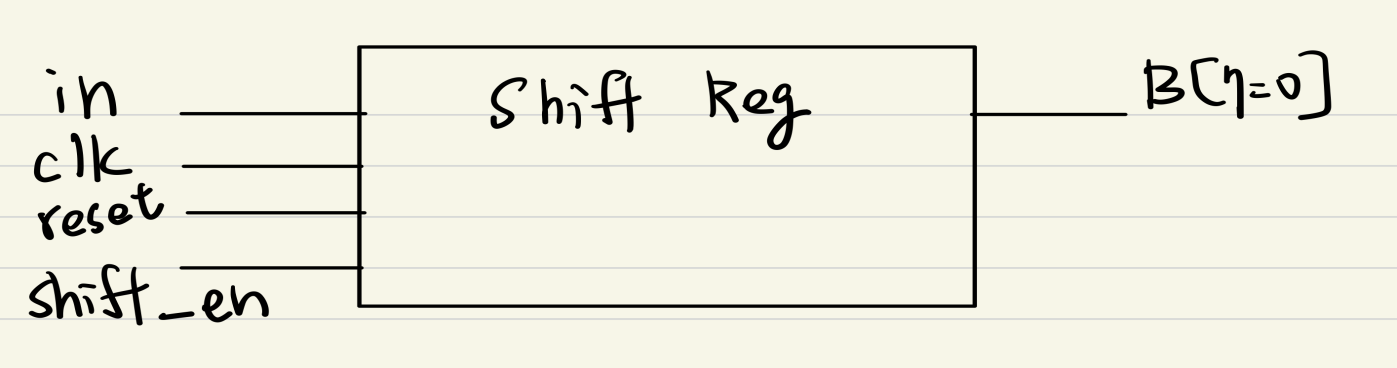
Output:

- B[7:0]: parallel output

Operation:

On each rising clock edge, when shift_en is asserted, the input bit is shifted into the register. After 8 clock cycles, the full 8-bit value is available at the output.

Block Diagram



Verilog Implementation & **Testbench**

tsenyuch@eng-analogsim3:~/ECE499/cadence/final_project/Multiplier599-

GNU nano 5.6.1 s2p_8bit.v

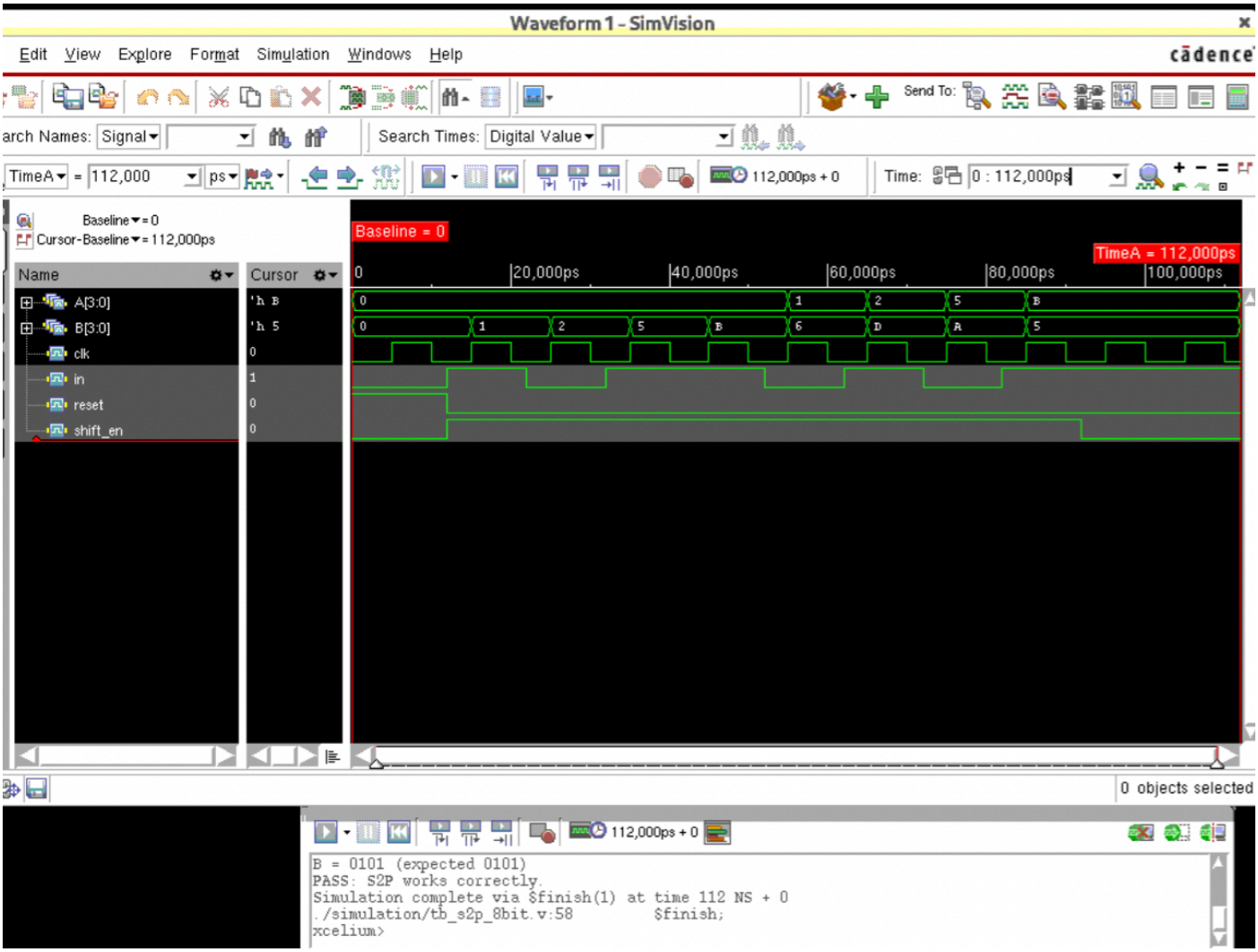
```
`timescale 1ns/1ps

module s2p_8bit (
    input       clk,
    input       reset,
    input       shift_en,
    input       in,
    output [3:0] A,
    output [3:0] B
);

    reg [7:0] shift_reg;

    always @(posedge clk or posedge reset) begin
        if (reset)
            shift_reg <= 8'b00000000;
        else if (shift_en)
            shift_reg <= {shift_reg[6:0], in};
    end

    assign A = shift_reg[7:4];
    assign B = shift_reg[3:0];

endmodule
```

tsenyuch@eng-ana

GNU nano 5.6.1

```
`timescale 1ns/1ps

module tb_s2p_8bit;

    reg        clk;
    reg        reset;
    reg        shift_en;
    reg        in;
    wire [3:0] A;
    wire [3:0] B;

    s2p_8bit uut (
        .clk(clk),
        .reset(reset),
        .shift_en(shift_en),
        .in(in),
        .A(A),
        .B(B)
    );

    // 10ns clock period
    always #5 clk = ~clk;

    initial begin
        clk      = 1'b0;
        reset    = 1'b1;
        shift_en = 1'b0;
        in       = 1'b0;

        // Reset
        #12;
        reset    = 1'b0;
        shift_en = 1'b1;

        // Shift in: 1011_0101
        // Expect A = 1011, B = 0101
        in = 1'b1; #10;
        in = 1'b0; #10;
        in = 1'b1; #10;
        in = 1'b1; #10;
        in = 1'b0; #10;
        in = 1'b1; #10;
        in = 1'b0; #10;
        in = 1'b1; #10;

        shift_en = 1'b0;
        #10;

        $display("A = %b (expected 1011)", A);
        $display("B = %b (expected 0101)", B);

        if (A == 4'b1011 && B == 4'b0101)
            $display("PASS: S2P works correctly.");
        else
```

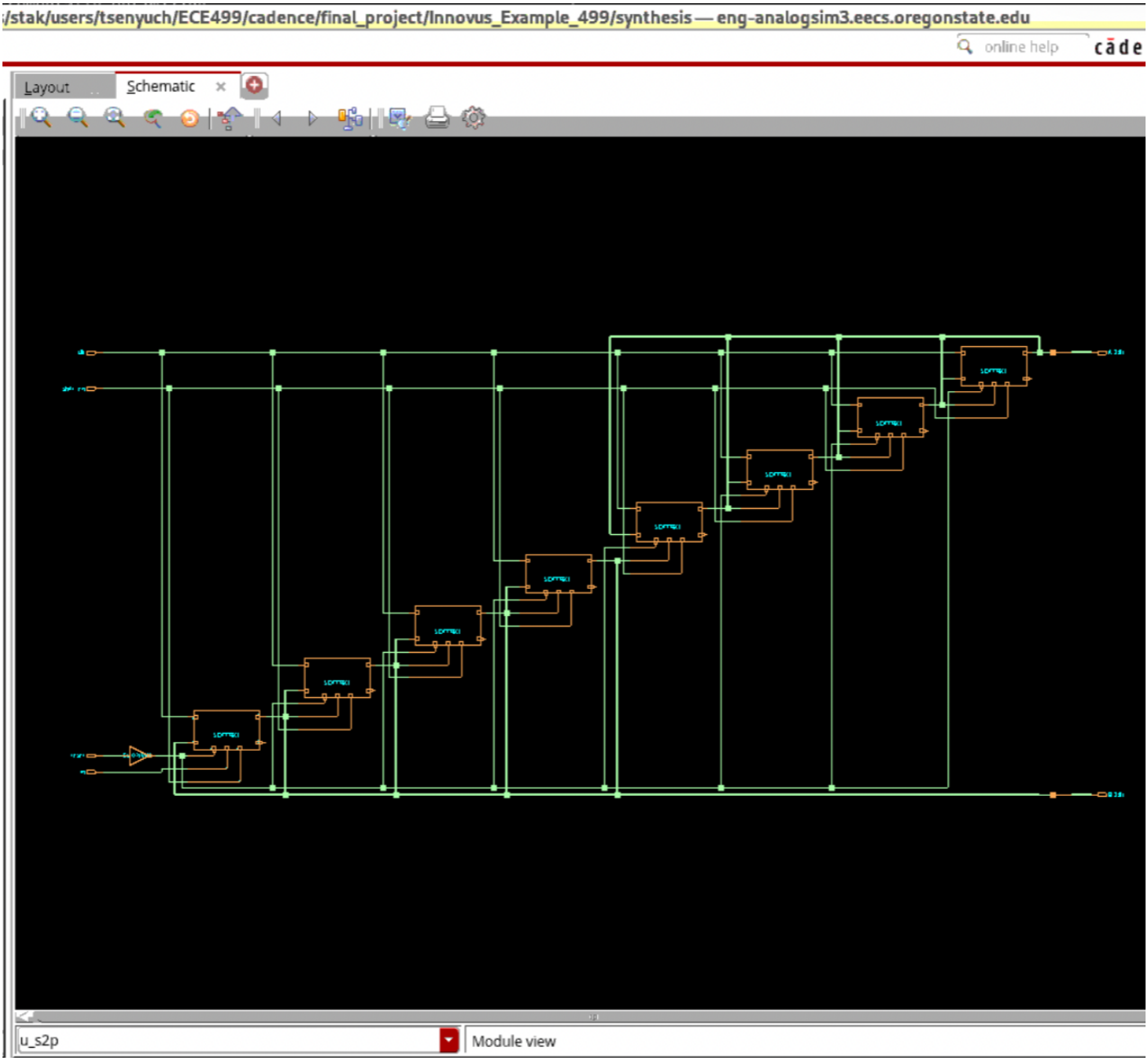
Simulation Results



The Serial-to-Parallel (S2P) converter was verified using Cadence Xcelium and SimVision. During simulation, serial input data was applied sequentially while the shift enable signal was asserted.

As shown in the waveform, the input bits are shifted into the register on each clock cycle. After the required number of cycles, the parallel output matches the expected value. For example, the final output B = 0101 confirms correct operation of the S2P module.

Synthesis Results (Genus)



2.3 8-bit Parallel-to-Serial

Functional Description:

The Parallel-to-Serial (P2S) module converts an 8-bit parallel input into a serial output stream.

Inputs:

- clk: system clock

- reset: clears the register

- load: loads parallel data

- shift_en: enables shifting

- P[7:0]: parallel input

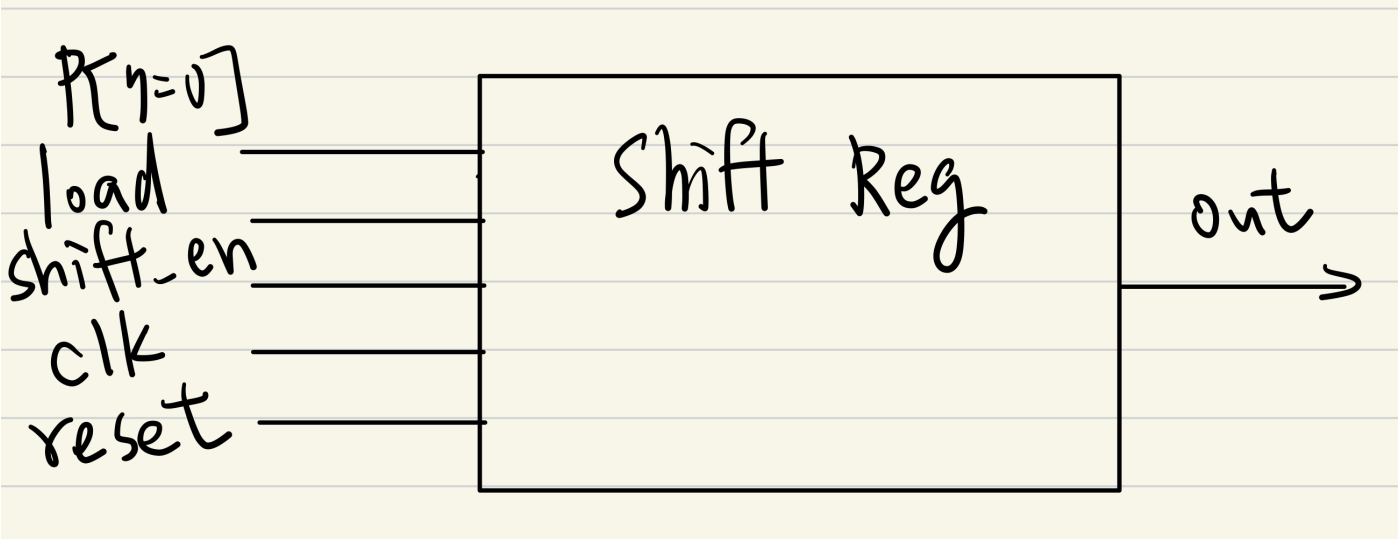
Output:

- out: serial output

Operation:

When load is asserted, the parallel input is stored into the register. When shift_en is enabled, the data is shifted out one bit per clock cycle, starting from the most significant bit.

Block Diagram



Verilog Implementation & Testbench

tsenyuch@eng-analogsim3:~/ECE499/cadence/final_project/Multiplier599-

GNU nano 5.6.1 p2s_8bit.v

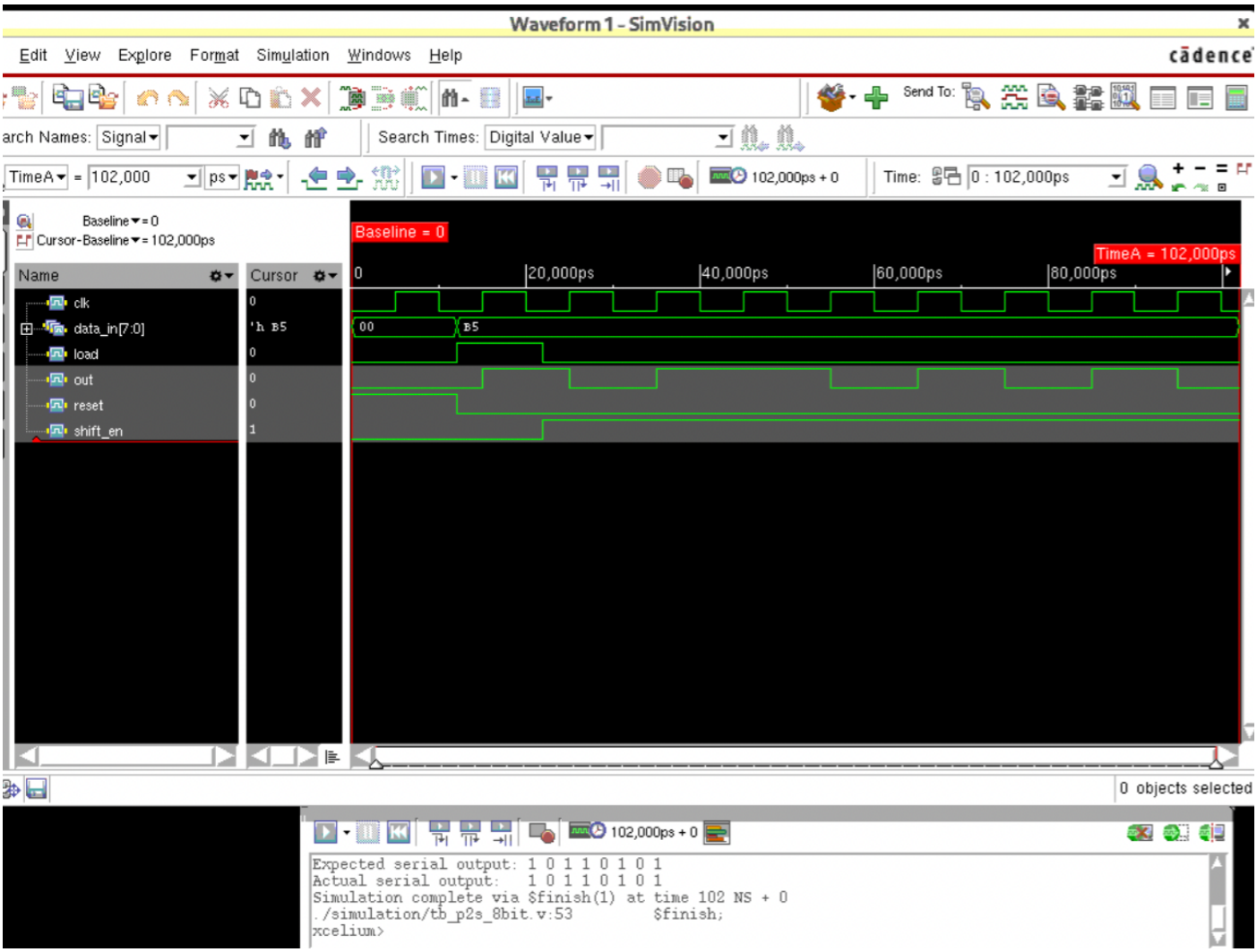
```
`timescale 1ns/1ps

module p2s_8bit (
    input       clk,
    input       reset,
    input       load,
    input       shift_en,
    input  [7:0] data_in,
    output      out
);

    reg [7:0] shift_reg;

    always @(posedge clk or posedge reset) begin
        if (reset)
            shift_reg <= 8'b0;
        else if (load)
            shift_reg <= data_in;
        else if (shift_en)
            shift_reg <= {shift_reg[6:0], 1'b0};
    end

    assign out = shift_reg[7];

endmodule
```

tsenyuch@eng-analogsim3:~

GNU nano 5.6.1

```
`timescale 1ns/1ps

module tb_p2s_8bit;

    reg clk;
    reg reset;
    reg load;
    reg shift_en;
    reg [7:0] data_in;
    wire out;

    p2s_8bit uut (
        .clk(clk),
        .reset(reset),
        .load(load),
        .shift_en(shift_en),
        .data_in(data_in),
        .out(out)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        load = 0;
        shift_en = 0;
        data_in = 8'b0;

        #12;
        reset = 0;

        // Load data = 10110101
        data_in = 8'b10110101;
        load = 1;
        #10;
        load = 0;

        // First bit should already be visible at out
        $display("Expected serial output: 1 0 1 1 0 1 0 1");
        $write("Actual serial output:   ");
        $write("%b ", out);

        shift_en = 1;

        repeat(7) begin
            #10;
            $write("%b ", out);
        end

        $display("");
        #10;
        $finish;
    end
```

Simulation Results

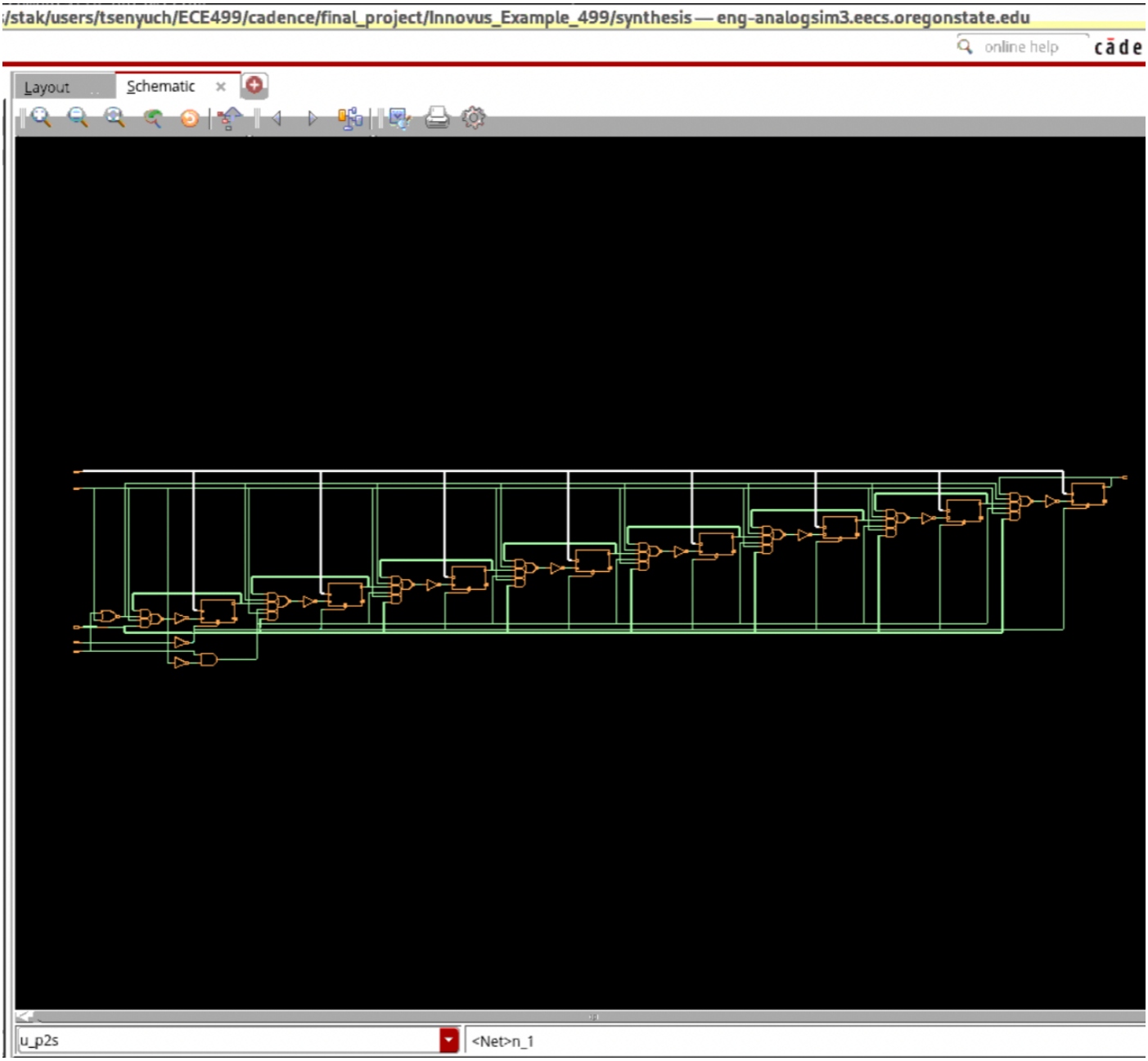


The Parallel-to-Serial (P2S) converter was verified using Cadence Xcelium and SimVision. A parallel input value was first loaded into the register when the load signal was asserted.

After loading, the shift enable signal was activated, and the data was shifted out serially on each clock cycle. The waveform shows that the output bits are produced sequentially and match the expected serial output.

For example, for an input value of 0xB5, the output sequence is 1 0 1 1 0 1 0 1, which matches the expected result, confirming correct operation of the P2S module.

Synthesis Results (Genus)



2.4 Controller FSM

Functional Description:

The controller is implemented as a Moore finite state machine (FSM) that manages the operation of the system.

Inputs:

- clk, reset, start

Outputs:

- s2p_shift_en

- p2s_load

- p2s_shift_en

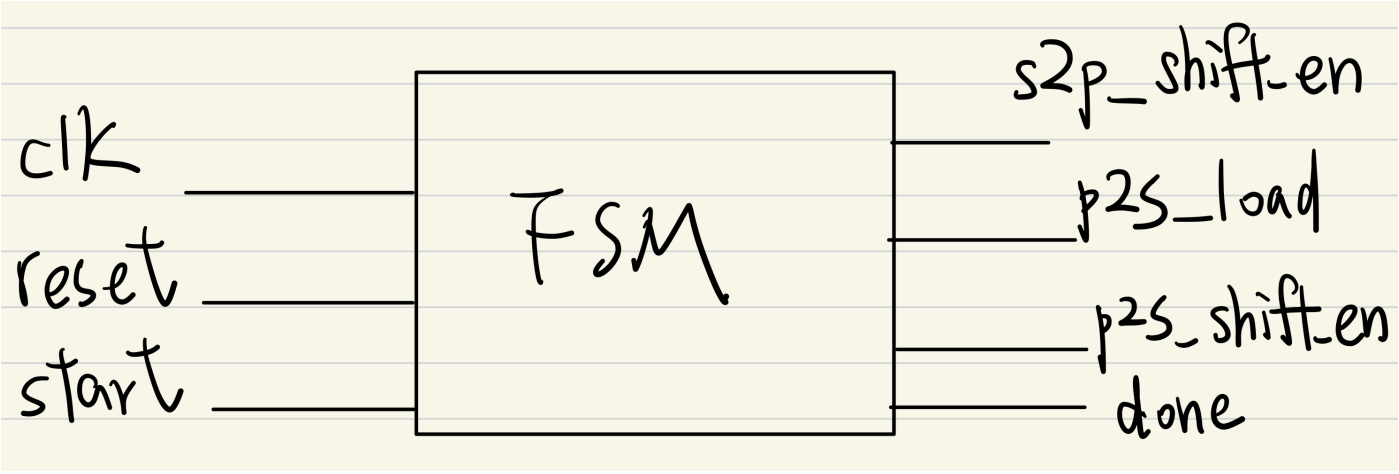
- done

Operation:

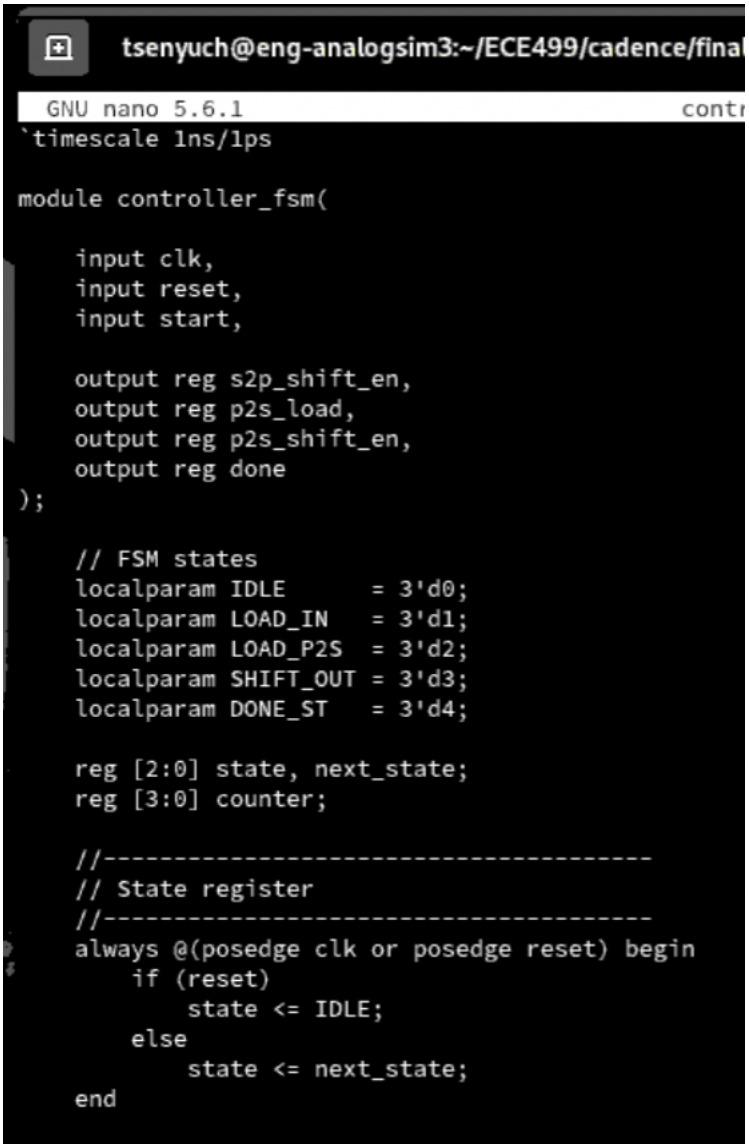
The FSM controls the sequence of operations, including loading input data, triggering the multiplier, and shifting out the result.

The system starts in the IDLE state. When start is asserted, the FSM enables input shifting (LOAD_IN), then loads the result into the output register (LOAD_P2S), and finally shifts out the result (SHIFT_OUT). The done signal is asserted at the end of the operation.

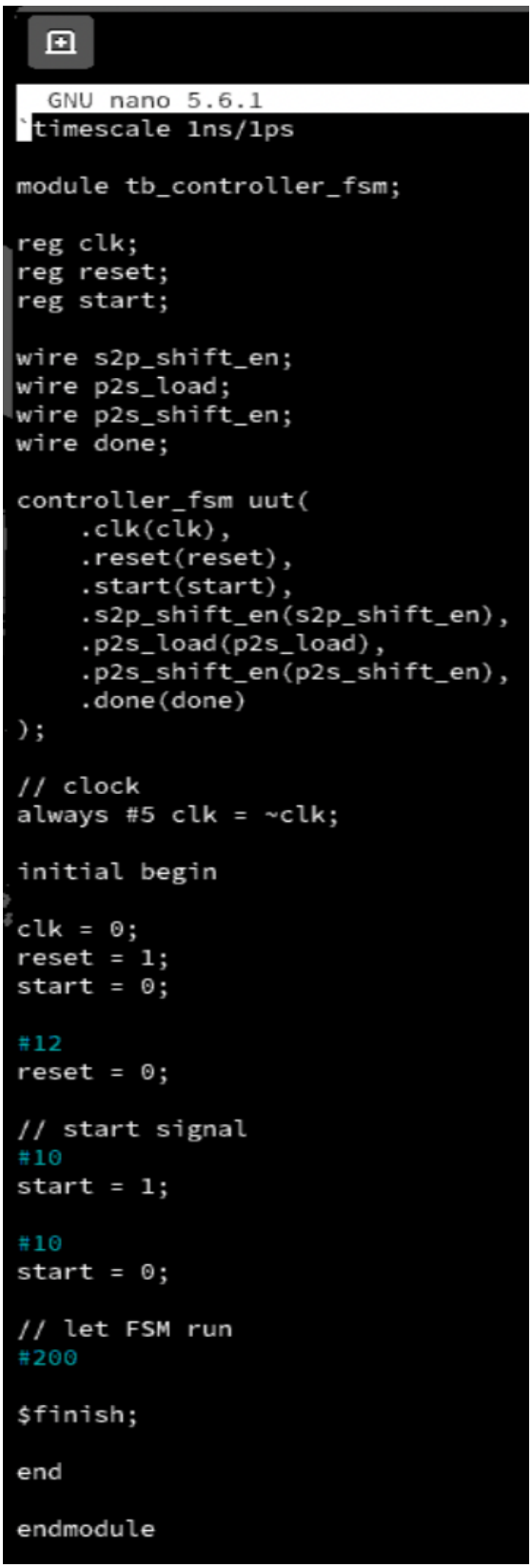
Block Diagram



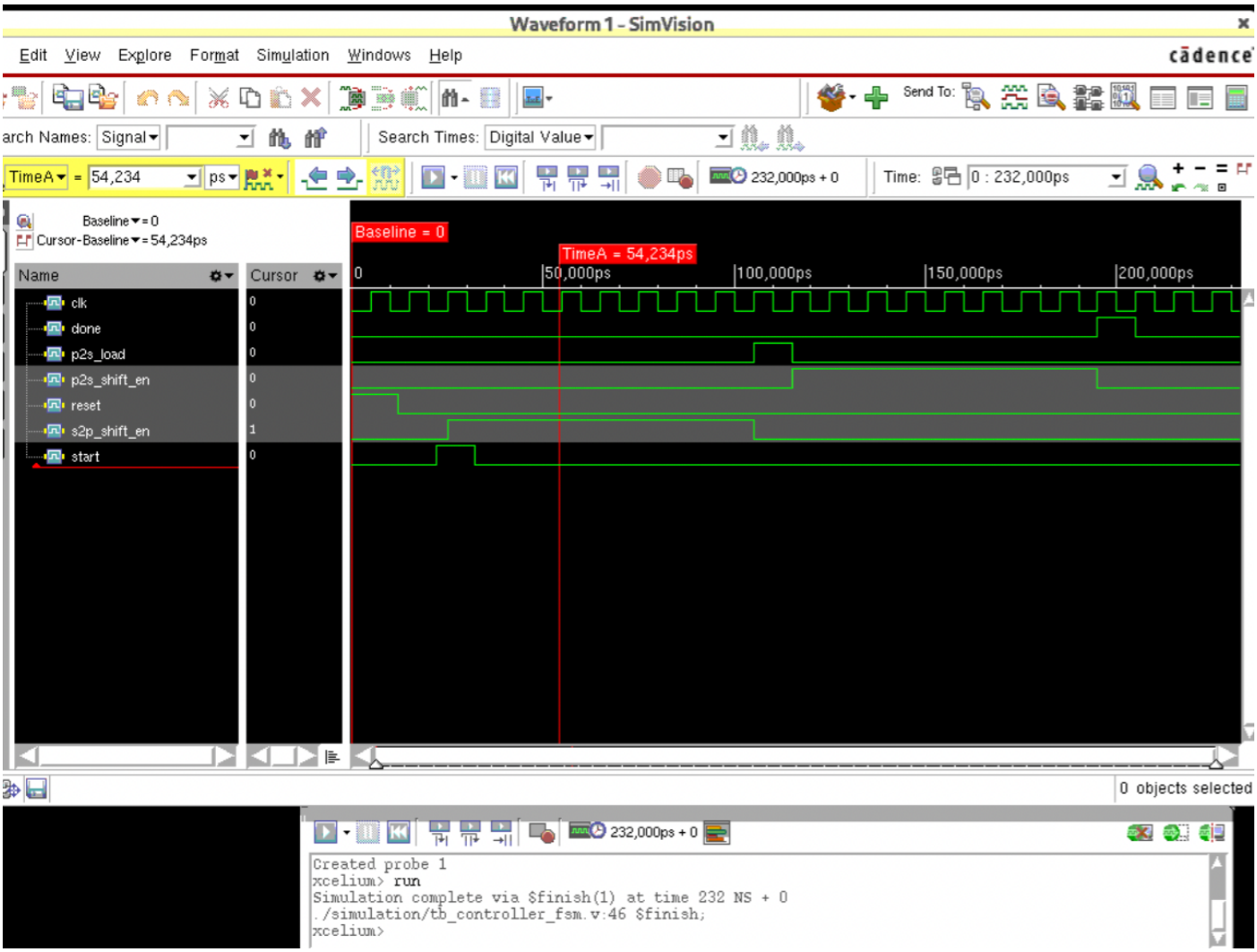
Verilog Implementation & Testbench



The Verilog implementation of the controller FSM is divided into three parts: the state register, the next-state logic, and the output logic. The state register is updated on the rising edge of the clock or reset, the next-state logic is implemented using a combinational case statement, and the control outputs are generated according to the active state. This structure improves readability and clearly separates sequential and combinational behavior.



Simulation Results



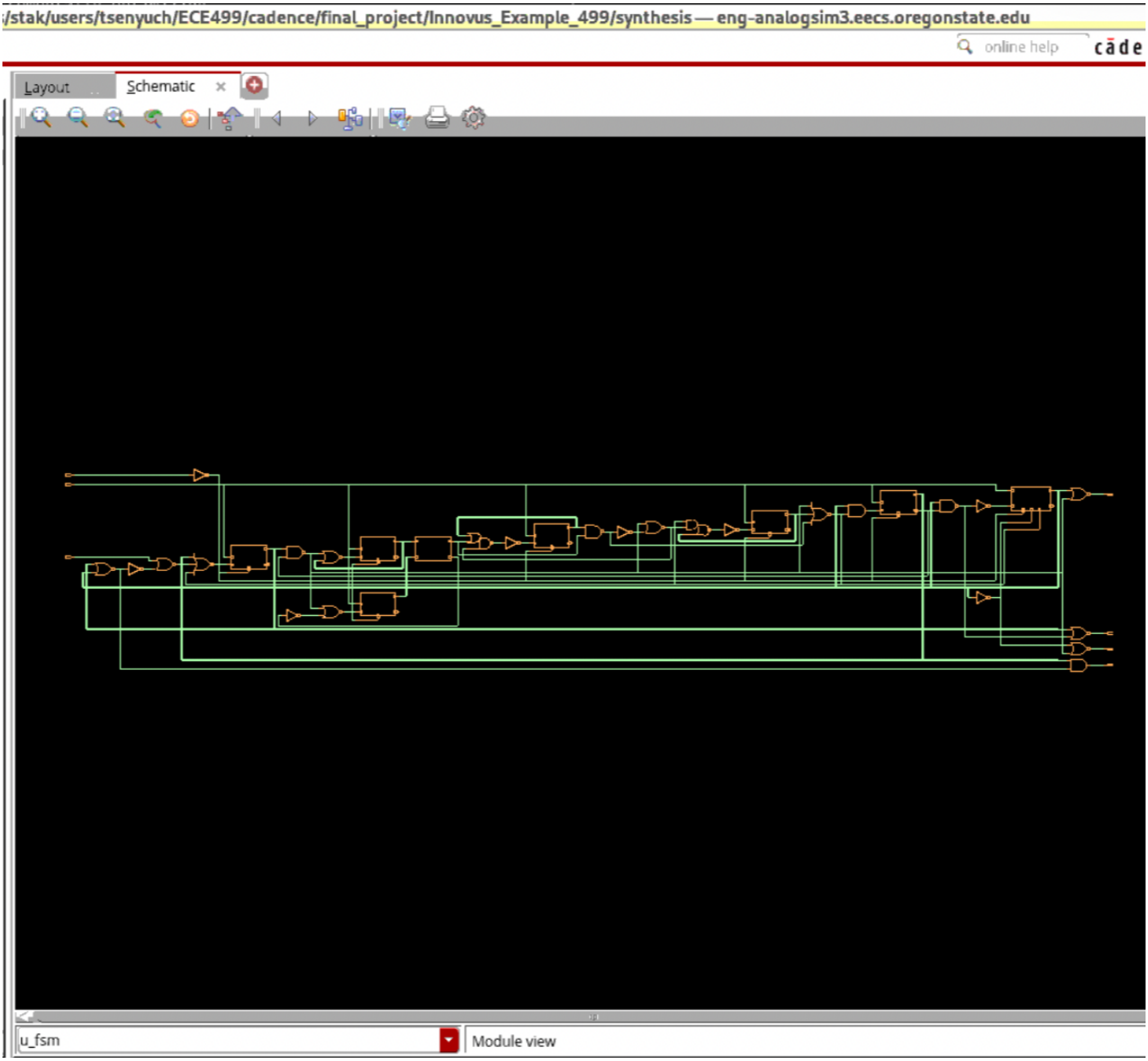
The controller FSM was verified using Cadence Xcelium and SimVision. The waveform demonstrates correct sequencing of control signals in response to the start signal.

Initially, the FSM remains in the IDLE state until the start signal is asserted. Upon receiving the start signal, the FSM enables the serial-to-parallel shift operation through s2p_shift_en. After the input data is fully loaded, the FSM transitions to the next stage, where p2s_load is asserted to load the parallel data into the output register.

Subsequently, the FSM enables p2s_shift_en to shift the data out serially. Finally, the done signal is asserted to indicate completion of the operation before returning to the IDLE state.

The waveform confirms that all control signals are activated in the correct order, demonstrating proper FSM operation.

Synthesis Results (Genus)



3. Top-Level Implementation

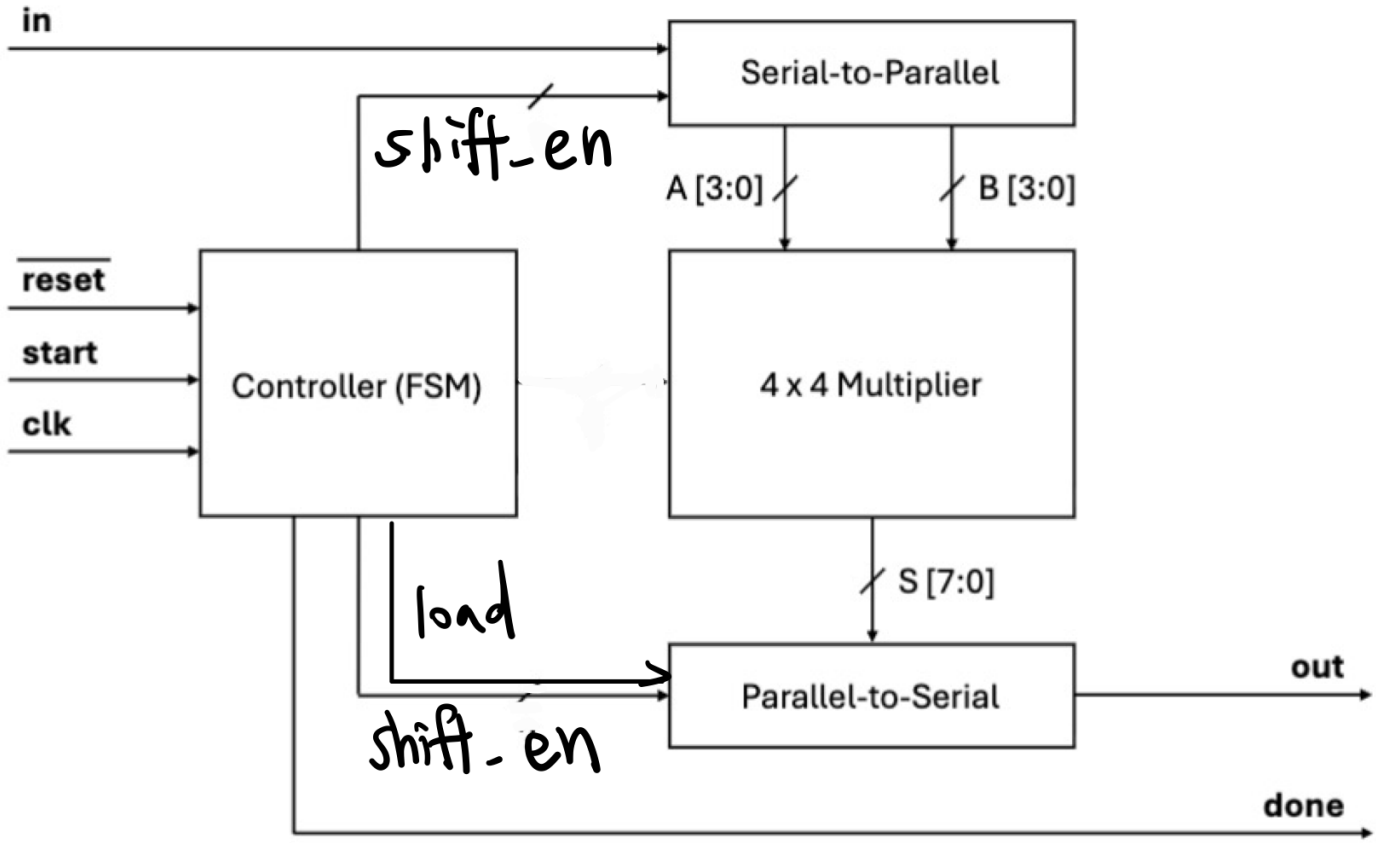
3.1 System Overview

The top-level system integrates the S2P module, 4×4 multiplier, P2S module, and FSM controller.

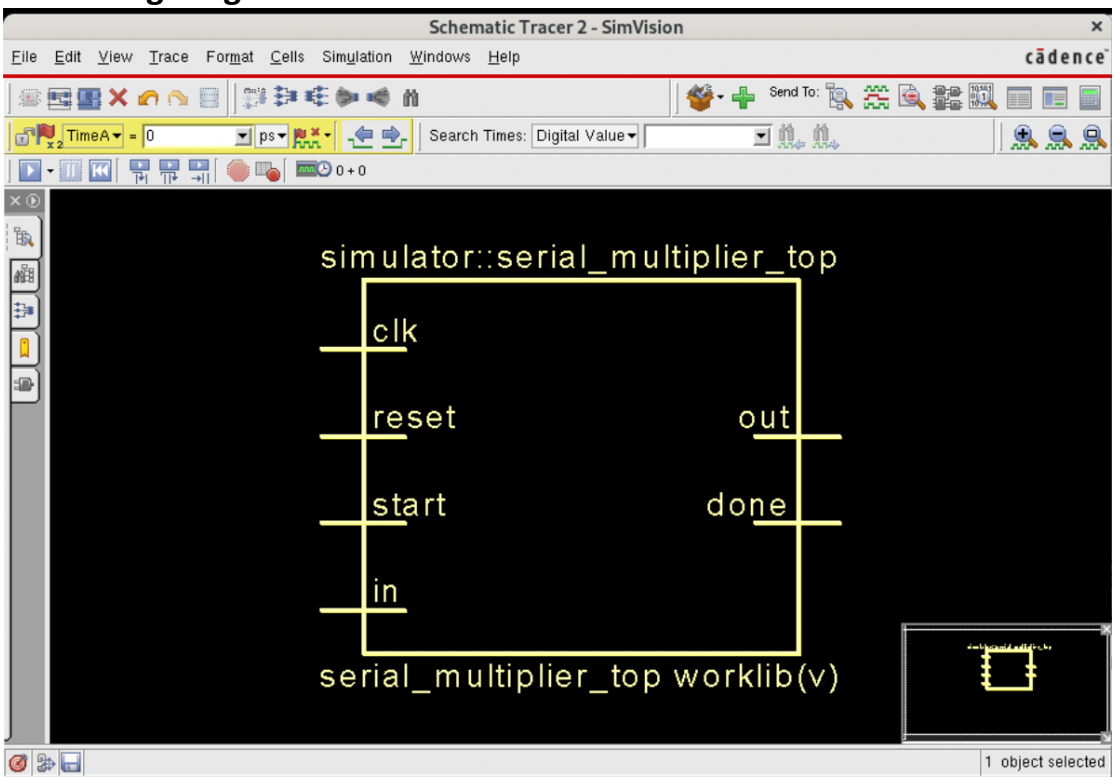
Serial input data is first converted into parallel form using the S2P block. The multiplier computes the result, which is then stored and shifted out serially using the P2S block.

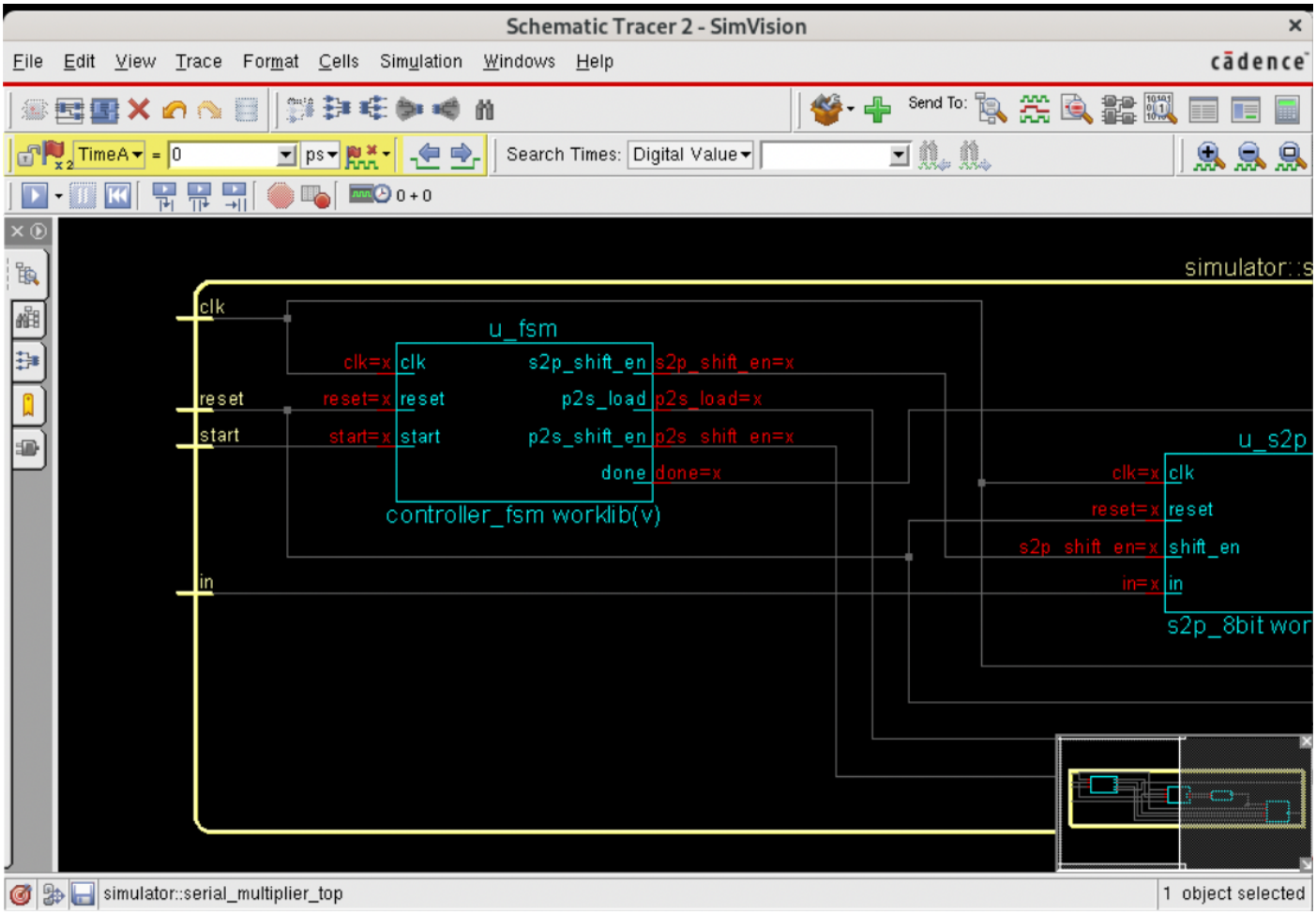
The FSM coordinates all operations by generating control signals to ensure correct sequencing of input loading, computation, and output shifting.

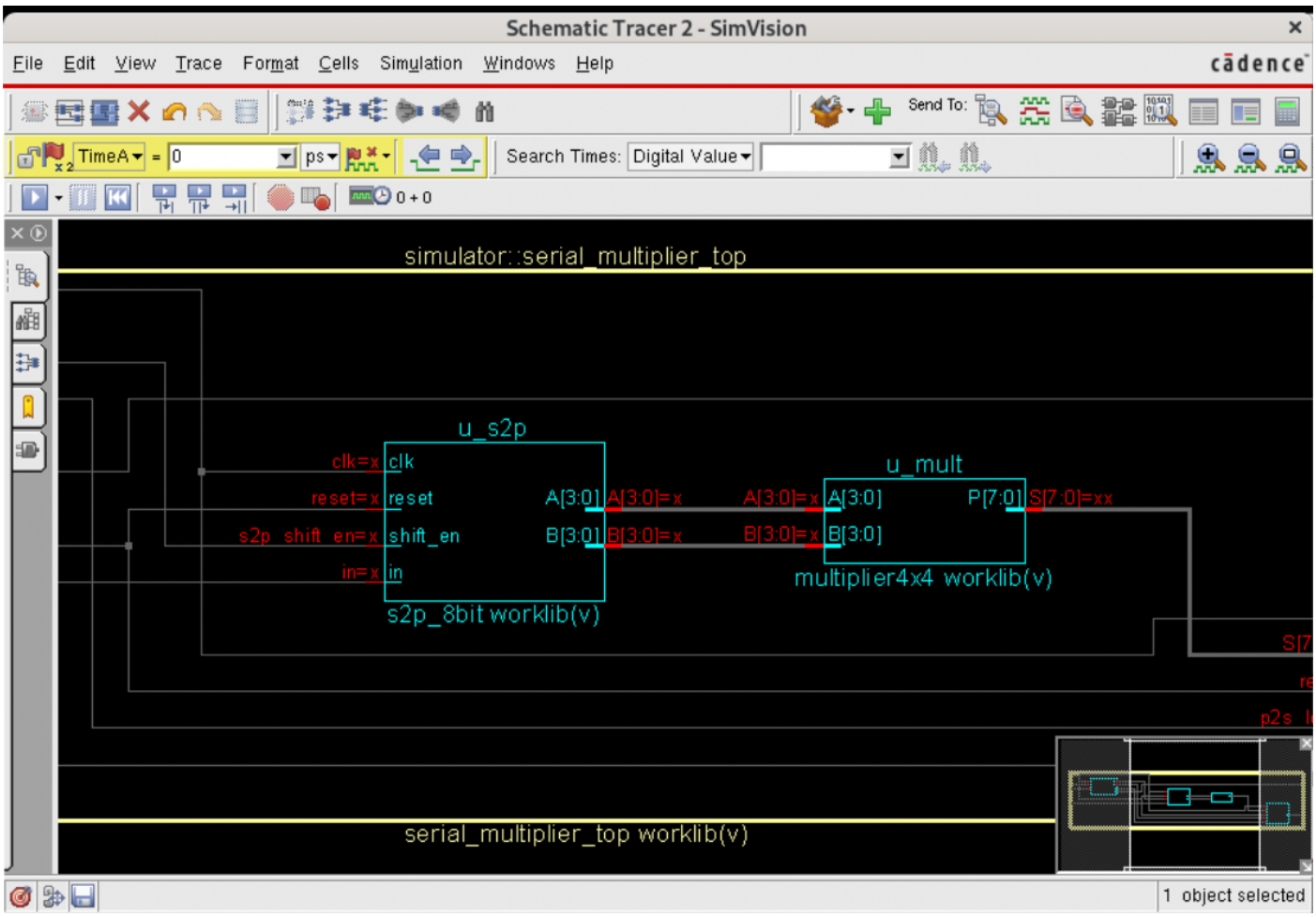
3.2 Block Diagram

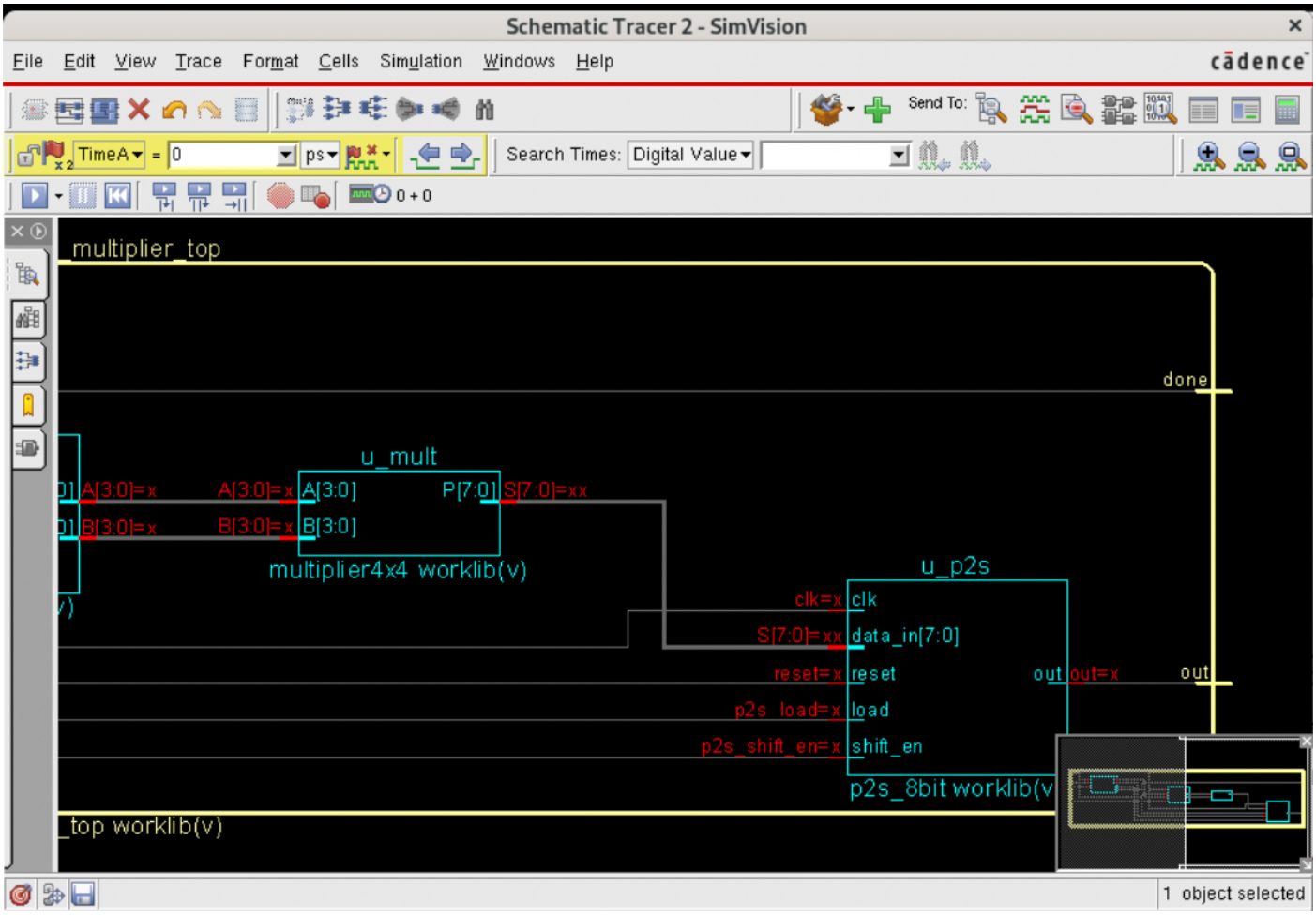


3.3 Timing Diagram




Schematic Tracer 2 - SimVision
File Edit View Trace Format Cells Simulation Windows Help
cadence
Send To:
TimeA = 0 ps
Search Times: Digital Value
0 + 0
simulator::s
clk
reset
start
in
u_fsm
clk=x clk
reset=x reset
start=x start
s2p_shift_en s2p_shift_en=x
p2s_load p2s_load=x
p2s_shift_en p2s_shift_en=x
done done=x
controller_fsm worklib(v)
u_s2p
clk=x clk
reset=x reset
s2p_shift_en=x shift_en
in=x in
s2p_8bit wor
simulator::serial_multiplier_top
1 object selected


Schematic Tracer 2 - SimVision
File Edit View Trace Format Cells Simulation Windows Help
cadence
Send To:
TimeA = 0 ps
Search Times: Digital Value
0 + 0
simulator::serial_multiplier_top
u_s2p
clk=x clk
reset=x reset
s2p_shift_en=x shift_en
in=x in
A[3:0] A[3:0]=x
B[3:0] B[3:0]=x
s2p_8bit worklib(v)
u_mult
A[3:0]=x A[3:0]
B[3:0]=x B[3:0]
P[7:0] S[7:0]=xx
multiplier4x4 worklib(v)
serial_multiplier_top worklib(v)
1 object selected


Schematic Tracer 2 - SimVision
File Edit View Trace Format Cells Simulation Windows Help
cadence
Send To:
TimeA = 0 ps
Search Times: Digital Value
0 + 0
multiplier_top
done
u_mult
A[3:0]=x
A[3:0]=x
A[3:0]
P[7:0]
S[7:0]=xx
B[3:0]=x
B[3:0]=x
B[3:0]
multiplier4x4 worklib(v)
u_p2s
clk=x clk
S[7:0]=xx data_in[7:0]
reset=x reset
out out=x
out
p2s_load=x load
p2s_shift_en=x shift_en
p2s_8bit worklib(v
_top worklib(v)
1 object selected

3.4 Verilog Implementation & Testbench

tsenyu

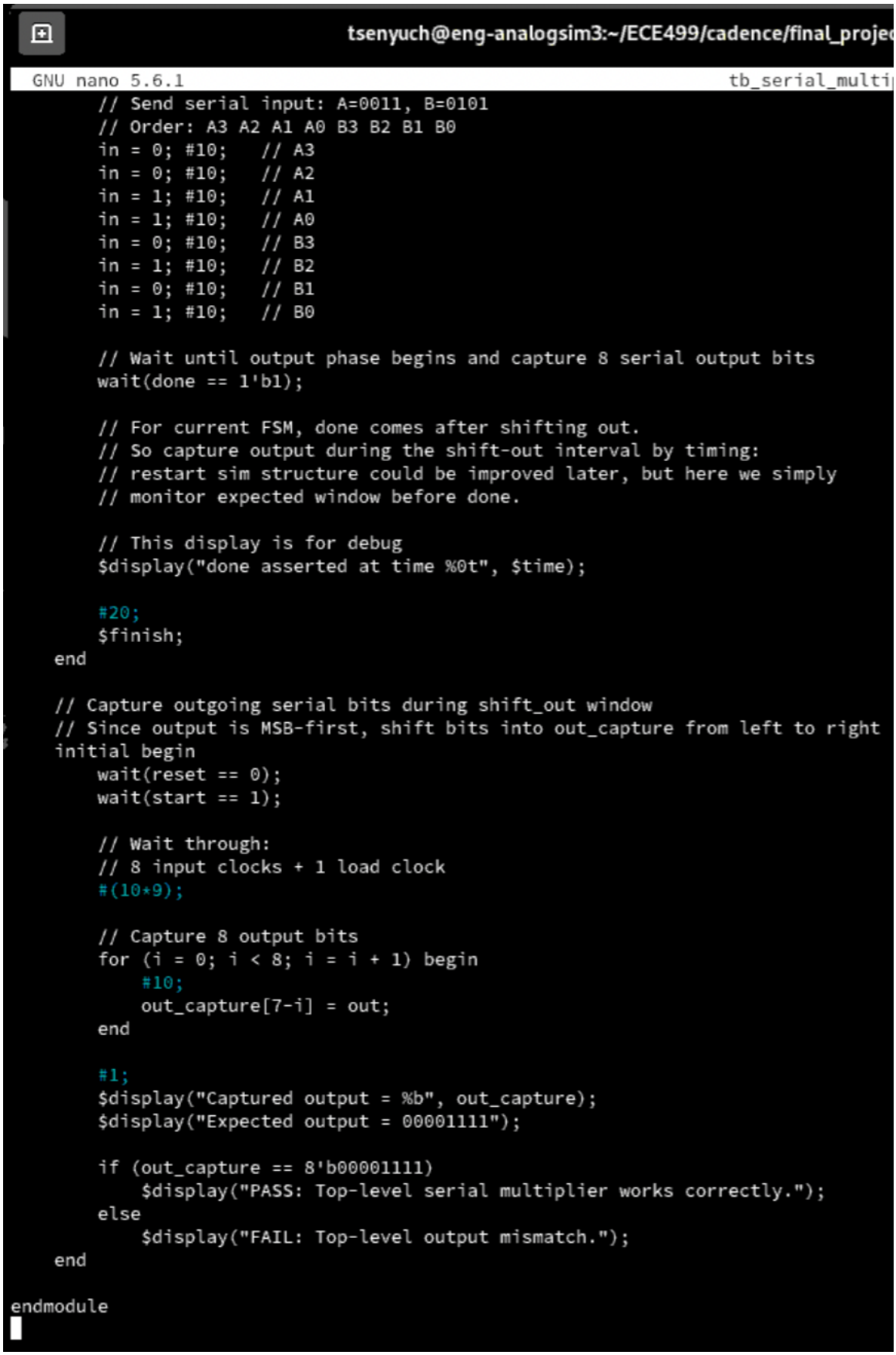
GNU nano 5.6.1

```
    input  clk,
    input  reset,
    input  start,
    input  in,
    output out,
    output done
);

    // datapath wires
    wire [3:0] A;
    wire [3:0] B;
    wire [7:0] S;

    // control wires
    wire s2p_shift_en;
    wire p2s_load;
    wire p2s_shift_en;

    // Serial ? Parallel
    s2p_8bit u_s2p (
        .clk(clk),
        .reset(reset),
        .shift_en(s2p_shift_en),
        .in(in),
        .A(A),
        .B(B)
    );

    // 4x4 multiplier
    multiplier4x4 u_mult (
        .A(A),
        .B(B),
        .P(S)
    );

    // Parallel ? Serial
    p2s_8bit u_p2s (
        .clk(clk),
        .reset(reset),
        .load(p2s_load),
        .shift_en(p2s_shift_en),
        .data_in(S),
        .out(out)
    );

    // Controller FSM
    controller_fsm u_fsm (
        .clk(clk),
        .reset(reset),
        .start(start),
        .s2p_shift_en(s2p_shift_en),
        .p2s_load(p2s_load),
        .p2s_shift_en(p2s_shift_en),
        .done(done)
```

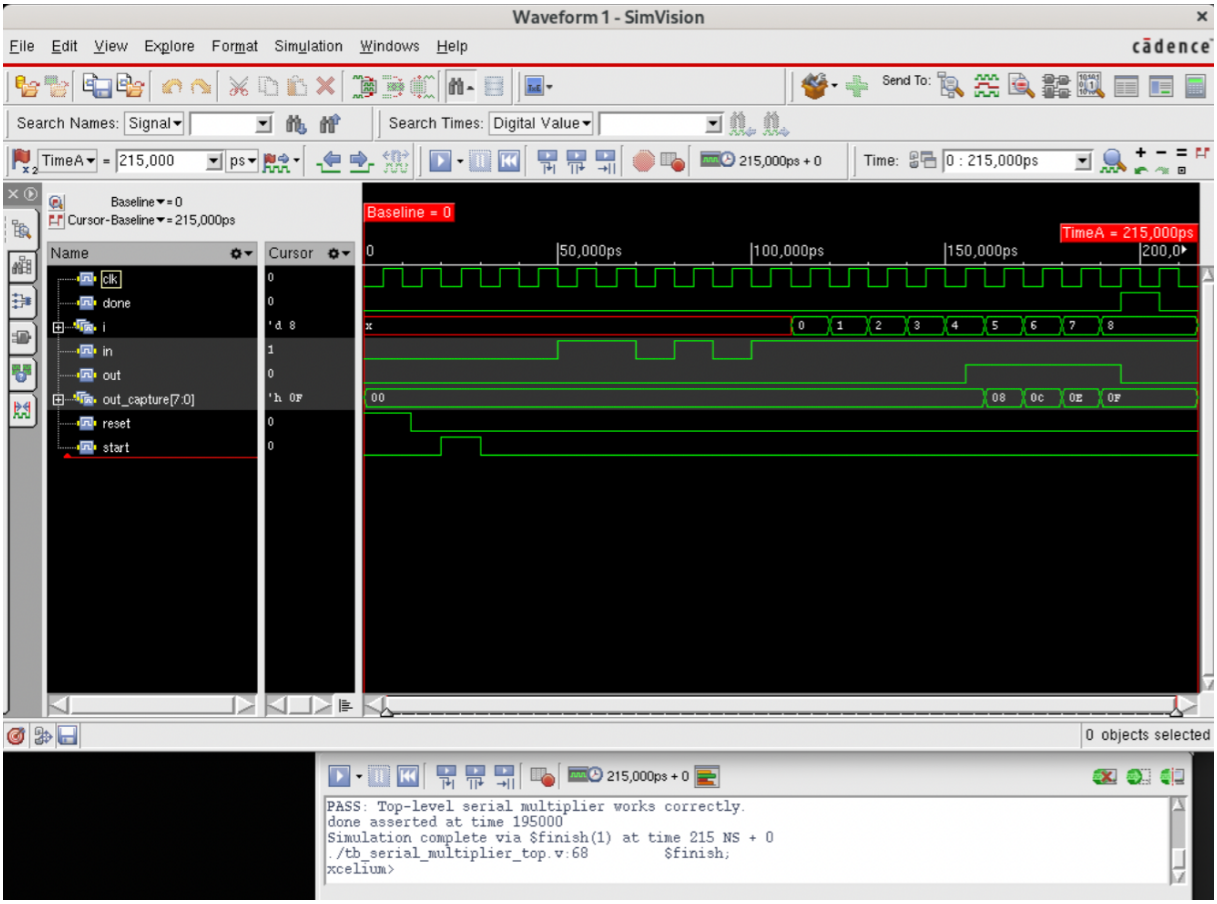
tsenyuch@eng-ar

GNU nano 5.6.1

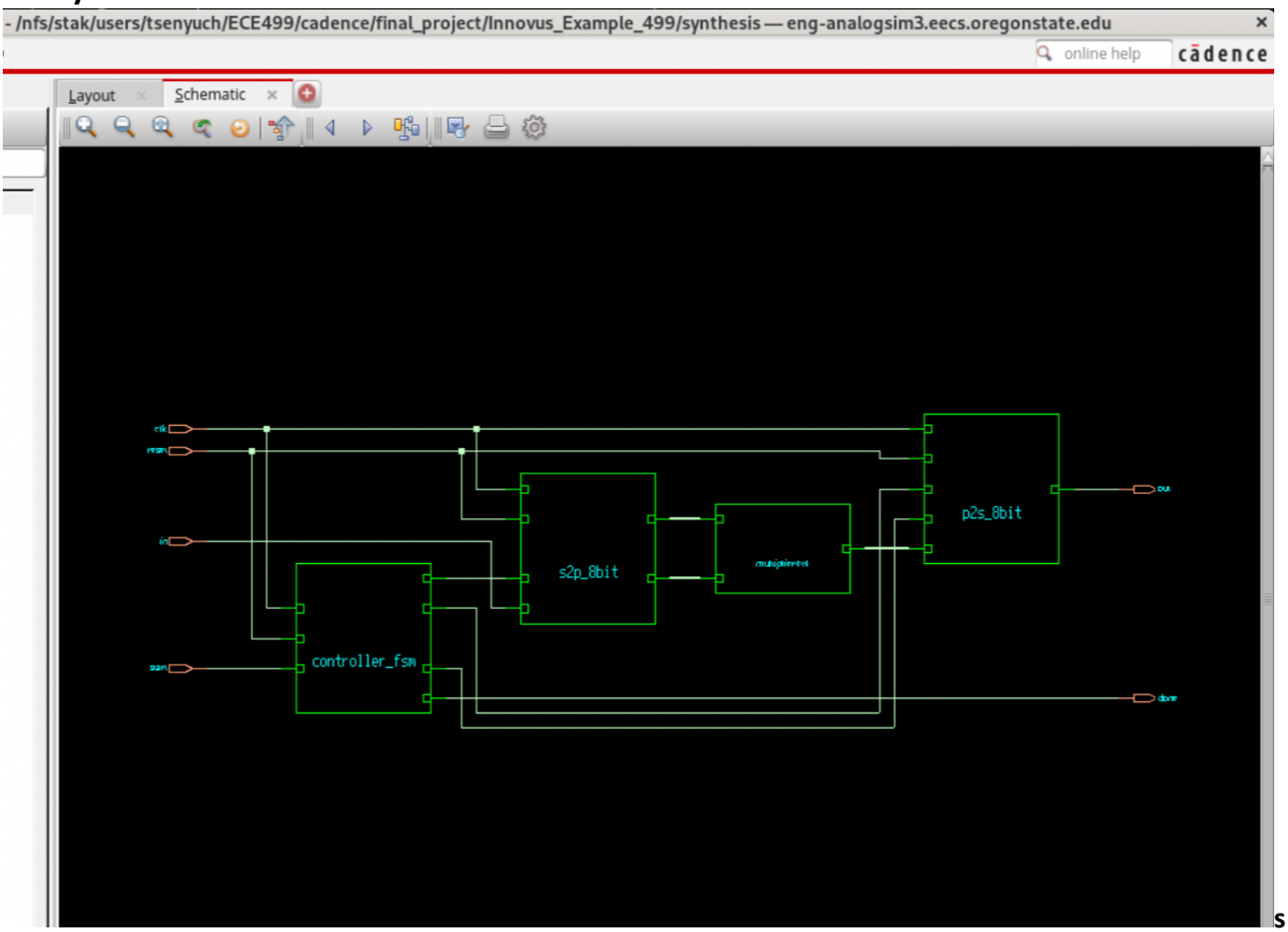
```
`timescale 1ns/1ps

module tb_serial_multiplier_top;

    reg clk;
    reg reset;
    reg start;
    reg in;

    wire out;
    wire done;

    reg [7:0] out_capture;
    integer i;

    serial_multiplier_top uut (
        .clk(clk),
        .reset(reset),
        .start(start),
        .in(in),
        .out(out),
        .done(done)
    );

    // 10ns clock period
    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        start = 0;
        in = 0;
        out_capture = 8'b0;

        // Reset
        #12;
        reset = 0;

        // Start pulse
        #8;
        start = 1;
        #10;
        start = 0;

        // Send serial input: A=0011, B=0101
        // Order: A3 A2 A1 A0 B3 B2 B1 B0
        in = 0; #10;   // A3
        in = 0; #10;   // A2
        in = 1; #10;   // A1
        in = 1; #10;   // A0
        in = 0; #10;   // B3
        in = 1; #10;   // B2
        in = 0; #10;   // B1
        in = 1; #10;   // B0
```


tsenyuch@eng-analogsim3:~/ECE499/cadence/final_proje
GNU nano 5.6.1
tb_serial_multi
// Send serial input: A=0011, B=0101
// Order: A3 A2 A1 A0 B3 B2 B1 B0
in = 0; #10; // A3
in = 0; #10; // A2
in = 1; #10; // A1
in = 1; #10; // A0
in = 0; #10; // B3
in = 1; #10; // B2
in = 0; #10; // B1
in = 1; #10; // B0
// Wait until output phase begins and capture 8 serial output bits
wait(done == 1'b1);
// For current FSM, done comes after shifting out.
// So capture output during the shift-out interval by timing:
// restart sim structure could be improved later, but here we simply
// monitor expected window before done.
// This display is for debug
$display("done asserted at time %0t", $time);
#20;
$finish;
end
// Capture outgoing serial bits during shift_out window
// Since output is MSB-first, shift bits into out_capture from left to right
initial begin
wait(reset == 0);
wait(start == 1);
// Wait through:
// 8 input clocks + 1 load clock
#(10*9);
// Capture 8 output bits
for (i = 0; i < 8; i = i + 1) begin
#10;
out_capture[7-i] = out;
end
#1;
$display("Captured output = %b", out_capture);
$display("Expected output = 00001111");
if (out_capture == 8'b00001111)
$display("PASS: Top-level serial multiplier works correctly.");
else
$display("FAIL: Top-level output mismatch.");
end
endmodule

3.5 Simulation Results



3.6 Synthesis Result



s

Timing report

```
tsenyuch@eng-analogsim3:~/ECE499/cadence/final_project/Multiplier59

====================================================================
  Generated by:           Genus(TM) Synthesis Solution 25.12-s067_1
  Generated on:           Mar 18 2026  10:28:44 pm
  Module:                 serial_multiplier_top
  Operating conditions:   ss_1.62_125
  Interconnect mode:      global
  Area mode:              physical library
====================================================================


Path 1: MET (4730 ps) Late External Delay Assertion at pin done
          Group: clk
     Startpoint: (R) u_fsm/state_reg[1]/CK
          Clock: (R) clk
       Endpoint: (F) done
          Clock: (R) clk

                     Capture       Launch
        Clock Edge:+   10000            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=   10000            0

      Output Delay:-    4000
       Uncertainty:-     150
     Required Time:=    5850
      Launch Clock:-       0
         Data Path:-    1120
             Slack:=    4730

Exceptions/Constraints:
  output_delay             4000            constraints_top.sdc_line_11

#-------------------------------------------------------------------------------------------------
#     Timing Point      Flags   Arc   Edge   Cell     Fanout Load Trans Delay Arrival Instance
#                                                        (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------------
  u_fsm/state_reg[1]/CK -       -     R     (arrival)     23    -   200     0       0    (-,-)
  u_fsm/state_reg[1]/Q  -       CK->Q R     SDFFRX1        5 22.8   264   750     750    (-,-)
  u_fsm/g592__6783/Y    -       B->Y  F     NOR2X1         2  8.3   104   166     916    (-,-)
  u_fsm/g587__6260/Y    -       B->Y  F     CLKAND2X2      1  1.8    34   204    1120    (-,-)
  done                  <<<     -     F     (port)         -    -     -     0    1120    (-,-)
#-------------------------------------------------------------------------------------------------
```

```
tsenyuch@eng-analogsim3:~/ECE499/cadence/fi

Instance: /serial_multiplier_top
Power Unit: W
PDB Frames: /stim#0/frame#0
  ---------------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ---------------------------------------------------------------------------
      memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
    register     1.87263e-07  6.34328e-05  8.37090e-06  7.19910e-05  59.36%
       latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       logic     6.76895e-08  1.86768e-05  1.96824e-05  3.84269e-05  31.68%
        bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       clock     0.00000e+00  0.00000e+00  1.08650e-05  1.08650e-05   8.96%
         pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
          pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ---------------------------------------------------------------------------
    Subtotal     2.54952e-07  8.21096e-05  3.89183e-05  1.21283e-04 100.00%
  Percentage           0.21%       67.70%       32.09%      100.00% 100.00%
  ---------------------------------------------------------------------------
```

Metric	Value
Max Delay	1.12 ns
Leakage Power	0.255 µW
Switching Power	38.9 µW
Total Power	121.3 µW

3.7 Maximum Clock Frequency

Worst-Case Delay Path and Maximum Clock Frequency

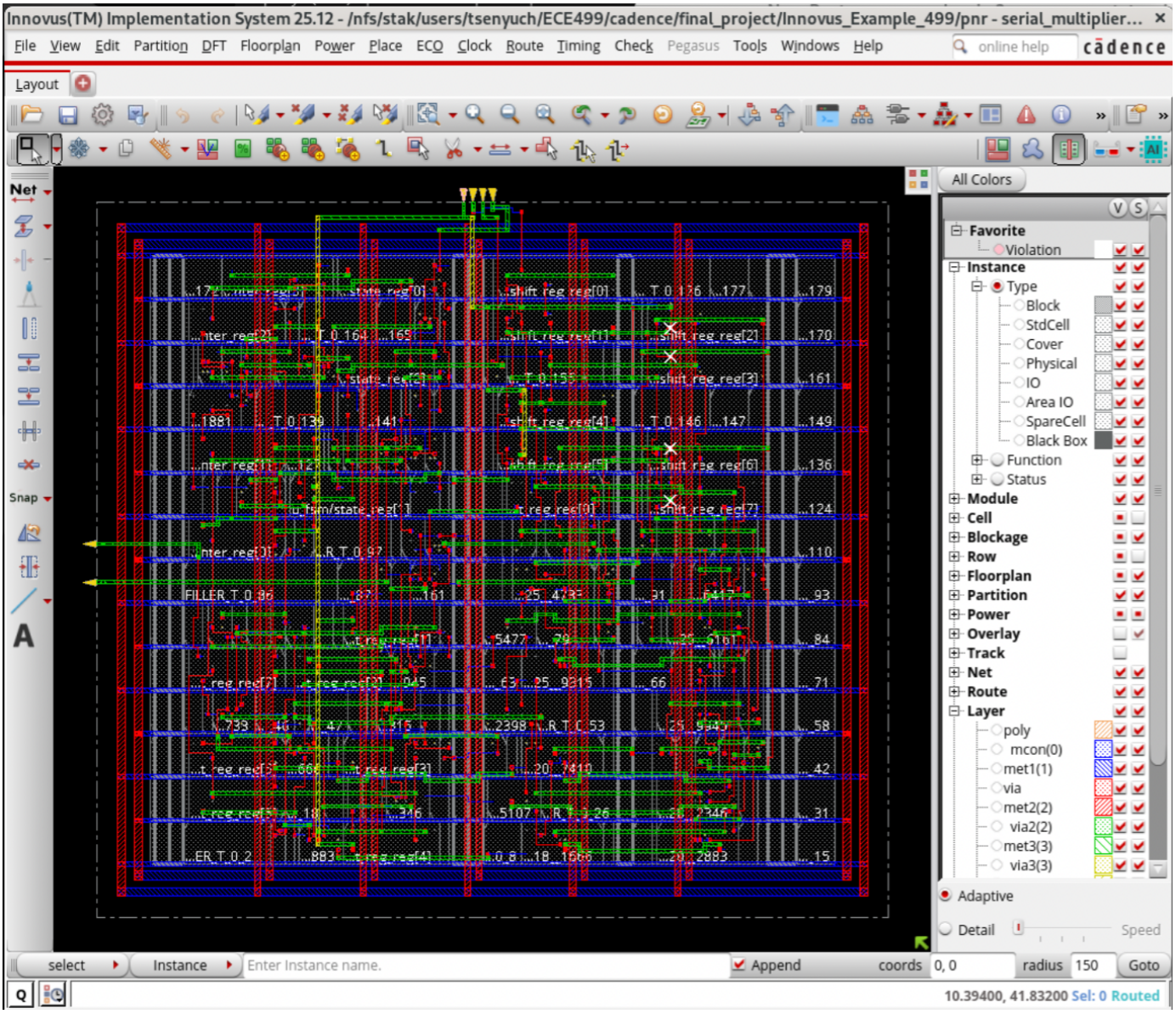
The post-synthesis timing report for the top-level design shows a worst-case path from u_fsm/state_reg[1]/CK to the output pin done. The reported data path delay for this path is 1120 ps, which corresponds to 1.12 ns. This value is taken as the critical path delay of the top-level system.

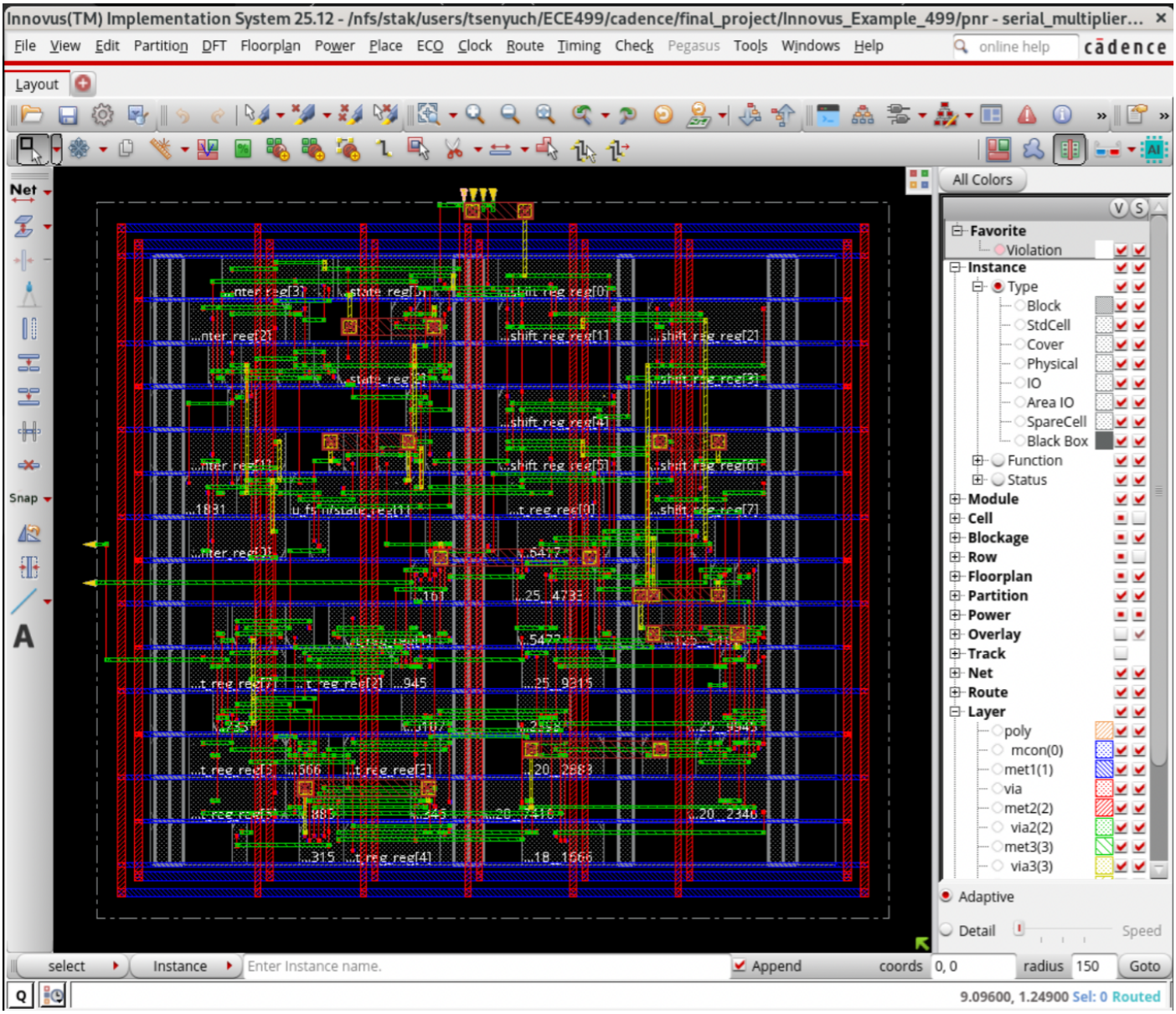
The maximum clock frequency is estimated from the critical path delay as:

$$f_{max} = \frac{1}{T_{critical}} = \frac{1}{1.12\ \text{ns}} \approx 892.9\ \text{MHz}$$

Therefore, the estimated maximum clock frequency of the top-level design is approximately 893 MHz.

3.8 Place and Route (Innovus)




Innovus(TM) Implementation System 25.12 - /nfs/stak/users/tsenyuch/ECE499/cadence/final_project/Innovus_Example_499/pnr - serial_multiplier...
File View Edit Partition DFT Floorplan Power Place ECO Clock Route Timing Check Pegasus Tools Windows Help
online help
cadence
Layout
Net
Snap
A
All Colors
Favorite
Violation
Instance
Type
Block
StdCell
Cover
Physical
IO
Area IO
SpareCell
Black Box
Function
Status
Module
Cell
Blockage
Row
Floorplan
Partition
Power
Overlay
Track
Net
Route
Layer
poly
mcon(0)
met1(1)
via
met2(2)
via2(2)
met3(3)
via3(3)
Adaptive
Detail
Speed
select
Instance
Enter Instance name.
Append
coords
0, 0
radius
150
Goto
9.09600, 1.24900 Sel: 0 Routed

```
Reset AAE Options
*** time_design #2 [finish] () : cpu/real = 0:00:00.6/0:00:00.9 (0.6), totSession cpu/real = 0:01
@innovus 102> set_db check_drc_disable_rules {} ; set_db check_drc_check_only default ; set_db ch
ck_reverse false ; set_db check_drc_short_only false ; set_db check_drc_check_routing_halo false
eck_drc_trim_length false ; set_db check_drc_uncolored false ; set_db check_drc_enable_post_passt
db check_drc_ignore_non_rectangle_shapes false ; set_db check_drc_inside_via_def true ; set_db ch
oute false ; set_db check_drc_ignore_cell_blockage false ; set_db check_drc_report {} ; set_db ch
@innovus 103> check_drc
#-check_same_via_cell true              # bool, default=false, user setting
#-report                                # string, default="", user setting
 *** Starting Verify DRC (MEM: 3310.9) ***

  VERIFY DRC ..... Starting Verification
  VERIFY DRC ..... Initializing
**WARN: (IMPVFG-1076):  Unable to create a report file. verify_drc report will not be generated.
  VERIFY DRC ..... Deleting Existing Violations
  VERIFY DRC ..... Creating Sub-Areas
  VERIFY DRC ..... Using new threading
  VERIFY DRC ..... Sub-Area: {0.000 0.000 75.440 68.080} 1 of 1
  VERIFY DRC ..... Sub-Area : 1 complete 4 Viols.

  Verification Complete : 4 Viols.

 Violation Summary By Layer and Type:

                  Short   Totals
       met2           4        4
       Totals         4        4

 *** End Verify DRC (CPU TIME: 0:00:00.0  ELAPSED TIME: 0:00:01.0  MEM: 10.6M) ***

@innovus 104> set_db check_drc_area {0 0 0 0}
@innovus 105>
```

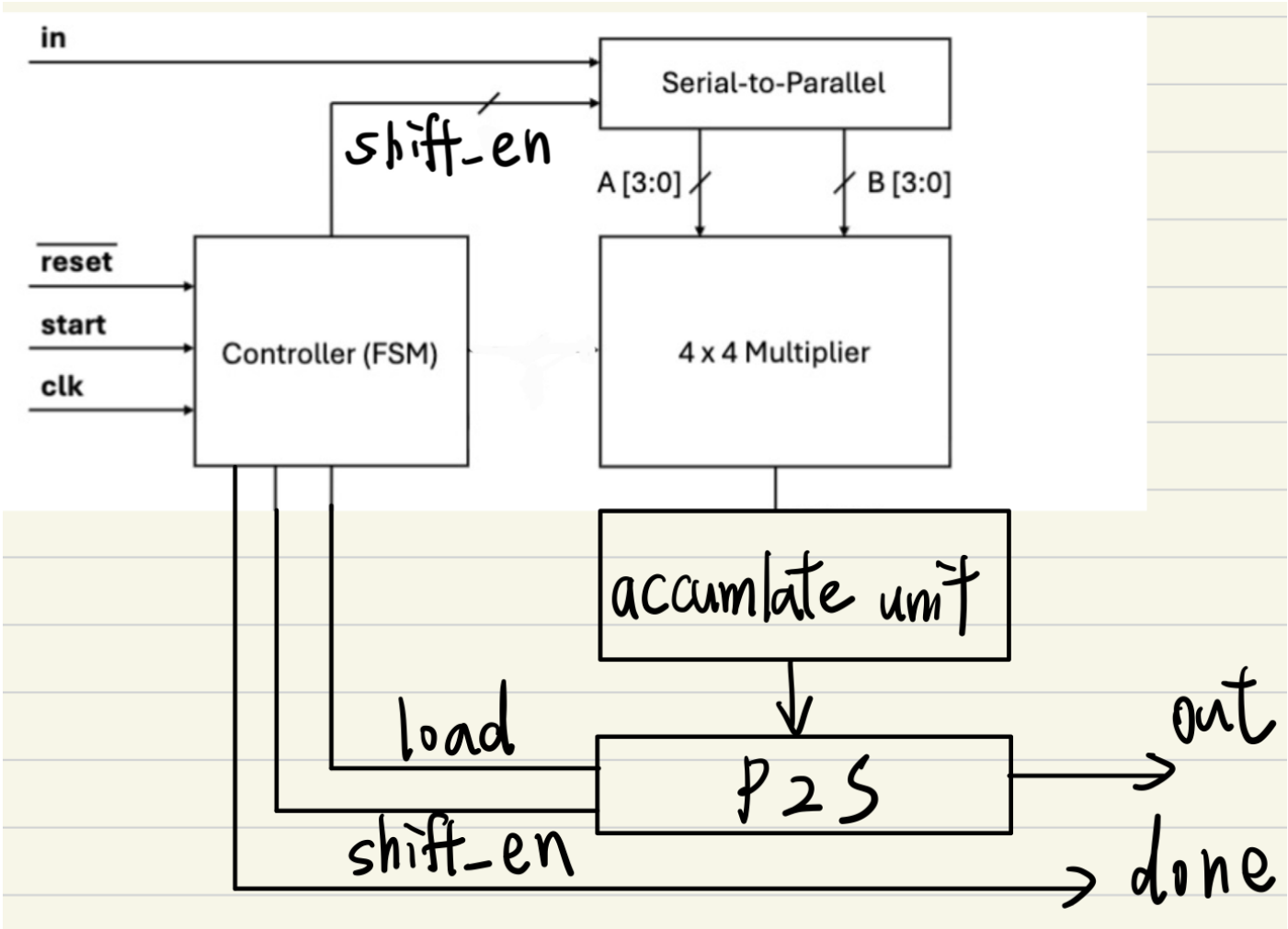
```
@innovus 104> set_db check_drc_area {0 0 0 0}
@innovus 105>
@innovus 105> check_connectivity -type all -geometry_connect -error 1000 -warning 50
VERIFY_CONNECTIVITY use new engine.

******** Start: VERIFY CONNECTIVITY ********
Start Time: Mon Mar  9 21:26:10 2026

Design Name: serial_multiplier_top
Database Units: 1000
Design Boundary: (0.0000, 0.0000) (75.4400, 68.0800)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Mon Mar  9 21:26:10 2026
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.062M)
```

4. Additional 599 Requirements

4.1 8-bit Multiplier Design

The 8-bit multiplier is implemented using a sequential architecture that reuses the existing 4×4 combinational multiplier to reduce hardware cost.
Instead of computing the full 8×8 multiplication in a single cycle, the operands are divided into 4-bit segments. The multiplication is performed over multiple cycles by generating partial products using the 4×4 multiplier and accumulating them with appropriate bit shifting.
At each step, a selected 4-bit portion of the operands is fed into the multiplier, and the resulting partial product is shifted according to its significance and added to an accumulator register. This process is repeated until all partial products have been generated and accumulated.
A finite state machine (FSM) controls the sequencing of operations, including input loading, partial-product generation, accumulation, and output shifting.
This approach significantly reduces area compared to a fully combinational 8×8 multiplier, at the cost of increased latency due to multi-cycle operation.

4.2 Block Diagram



tsenyuch@eng-analogsim3:

GNU nano 5.6.1

```
`timescale 1ns/1ps

module serial_multiplier8x8_top (
    input  clk,
    input  reset,
    input  start,
    input  in,
    output out,
    output done
);

    // ----------------------------
    // Internal datapath wires
    // ----------------------------
    wire [7:0] A;
    wire [7:0] B;

    wire [3:0] mult_a;
    wire [3:0] mult_b;
    wire [7:0] mult_out;

    wire [15:0] acc_out;

    // ----------------------------
    // Internal control wires
    // ----------------------------
    wire        s2p_shift_en;
    wire        clear_acc;
    wire        load_acc;
    wire        p2s_load;
    wire        p2s_shift_en;
    wire [1:0]  shift_sel;
    wire        a_sel;
    wire        b_sel;

    // ----------------------------
    // Serial-to-Parallel 16-bit
    // ----------------------------
    s2p_16bit u_s2p (
        .clk(clk),
        .reset(reset),
        .shift_en(s2p_shift_en),
        .in(in),
        .A(A),
        .B(B)
    );

    // ----------------------------
    // Nibble select
    // a_sel = 0 -> A_lo, 1 -> A_hi
    // b_sel = 0 -> B_lo, 1 -> B_hi
    // ----------------------------
    assign mult_a = (a_sel == 1'b0) ? A[3:0] : A[7:4];
    assign mult_b = (b_sel == 1'b0) ? B[3:0] : B[7:4];
```

tsenyuch@eng-analogsim3:

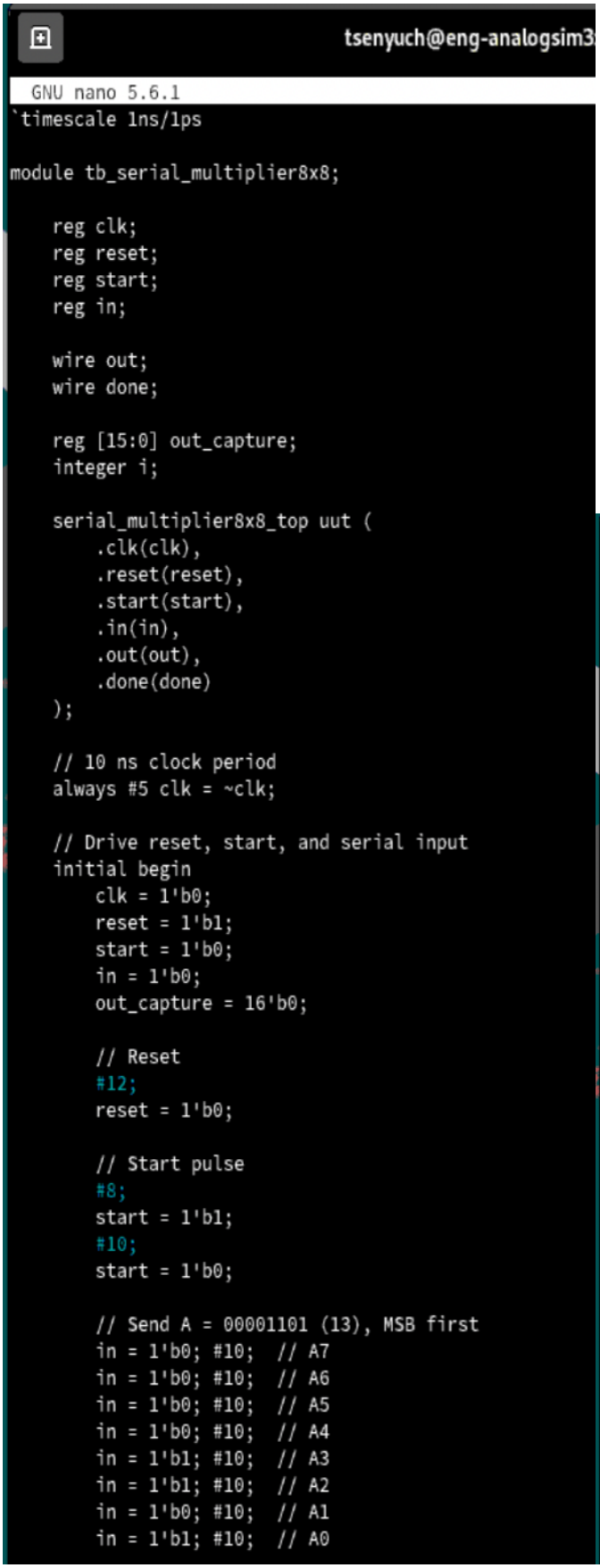
GNU nano 5.6.1

```
    // ----------------------------
    // 4x4 multiplier
    // ----------------------------
    multiplier4x4 u_mult (
        .A(mult_a),
        .B(mult_b),
        .P(mult_out)
    );

    // ----------------------------
    // Accumulate unit
    // ----------------------------
    accumulate_unit u_acc (
        .clk(clk),
        .reset(reset),
        .clear_acc(clear_acc),
        .load_acc(load_acc),
        .mult_out(mult_out),
        .shift_sel(shift_sel),
        .acc_out(acc_out)
    );

    // ----------------------------
    // Parallel-to-Serial 16-bit
    // ----------------------------
    p2s_16bit u_p2s (
        .clk(clk),
        .reset(reset),
        .load(p2s_load),
        .shift_en(p2s_shift_en),
        .data_in(acc_out),
        .out(out)
    );

    // ----------------------------
    // FSM controller
    // ----------------------------
    controller_fsm_599 u_fsm (
        .clk(clk),
        .reset(reset),
        .start(start),
        .s2p_shift_en(s2p_shift_en),
        .clear_acc(clear_acc),
        .load_acc(load_acc),
        .p2s_load(p2s_load),
        .p2s_shift_en(p2s_shift_en),
        .done(done),
        .shift_sel(shift_sel),
        .a_sel(a_sel),
        .b_sel(b_sel)
    );

endmodule
```

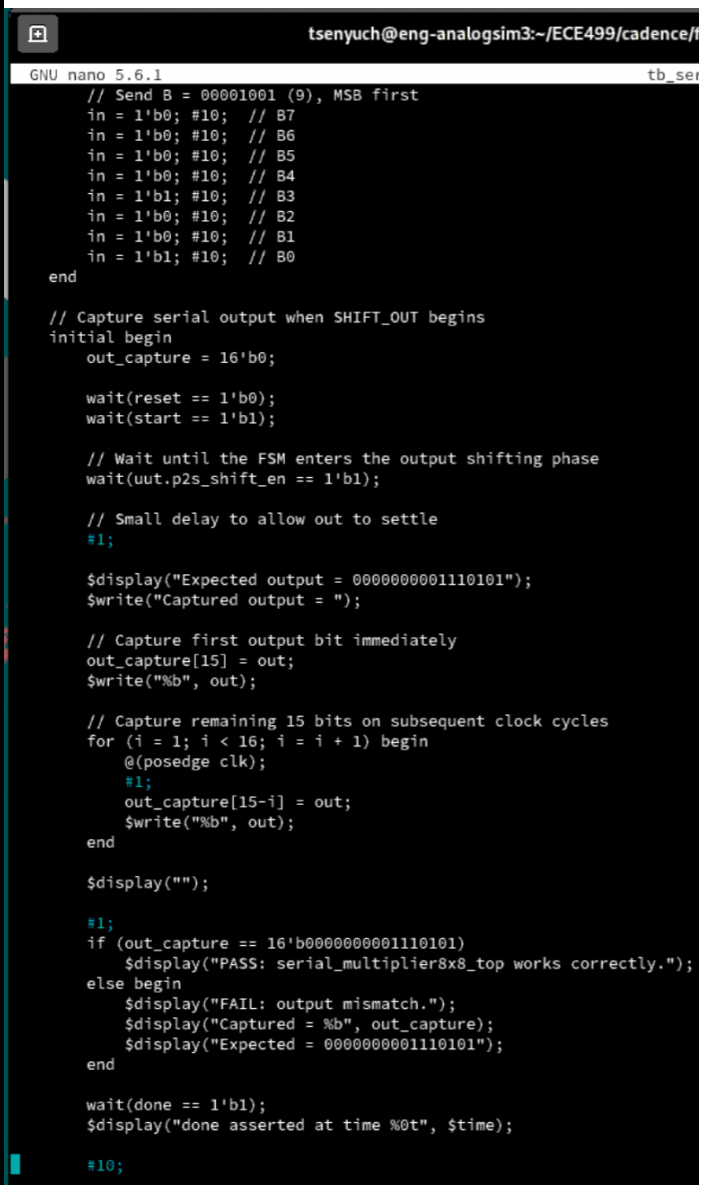
4.3 Verilog Implementation

The `serial_multiplier8x8_top` module serves as the structural integration layer of the entire 8×8 serial multiplier. It connects the input/output conversion blocks, the reused 4×4 multiplier, the accumulation unit, and the FSM controller. The FSM determines which 4-bit portions of the operands are sent into the multiplier, how much each partial product should be shifted, and when the accumulated result should be loaded and shifted out. In this way, the top-level module coordinates all submodules into a complete

sequential multiplier system.



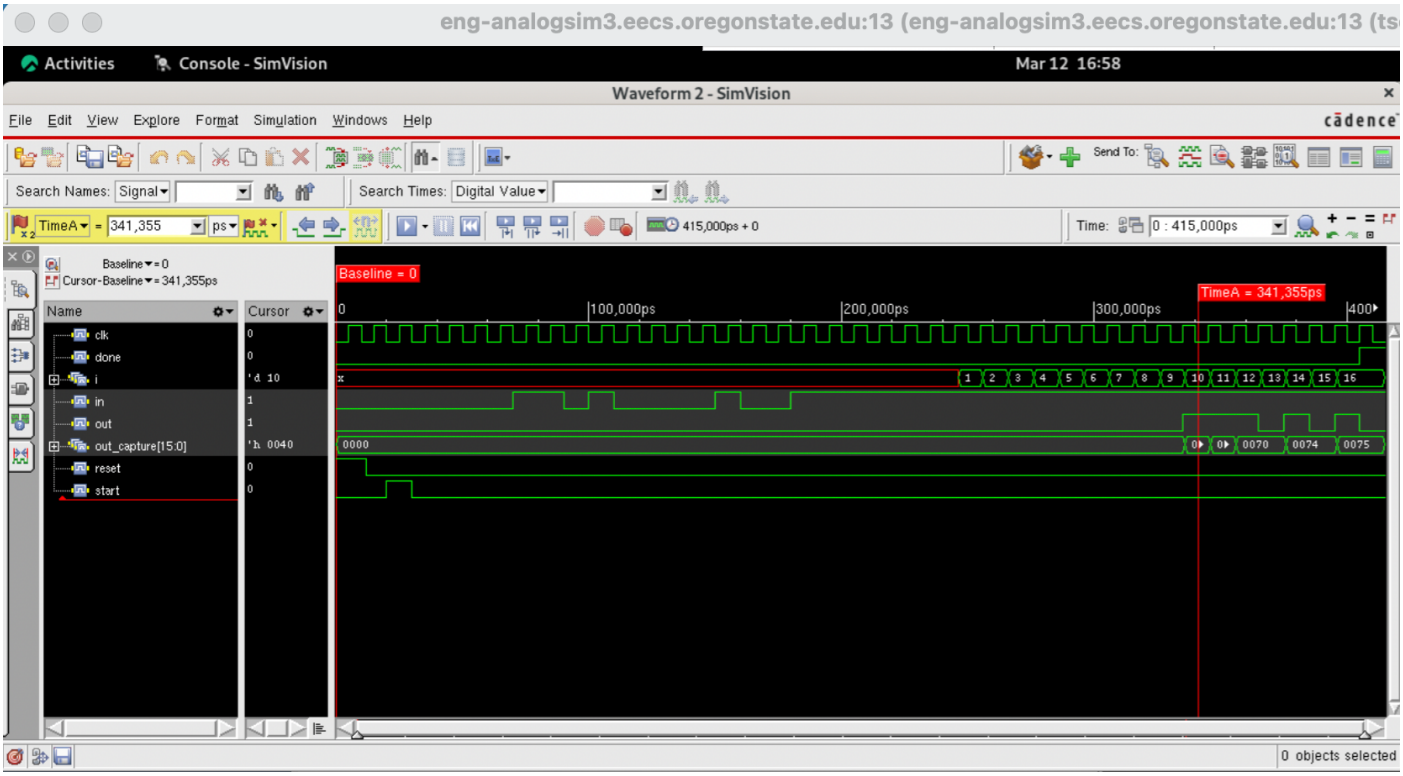
```
tsenyuch@eng-analogsim3
 GNU nano 5.6.1
`timescale 1ns/1ps

module tb_serial_multiplier8x8;

    reg clk;
    reg reset;
    reg start;
    reg in;

    wire out;
    wire done;

    reg [15:0] out_capture;
    integer i;

    serial_multiplier8x8_top uut (
        .clk(clk),
        .reset(reset),
        .start(start),
        .in(in),
        .out(out),
        .done(done)
    );

    // 10 ns clock period
    always #5 clk = ~clk;

    // Drive reset, start, and serial input
    initial begin
        clk = 1'b0;
        reset = 1'b1;
        start = 1'b0;
        in = 1'b0;
        out_capture = 16'b0;

        // Reset
        #12;
        reset = 1'b0;

        // Start pulse
        #8;
        start = 1'b1;
        #10;
        start = 1'b0;

        // Send A = 00001101 (13), MSB first
        in = 1'b0; #10;  // A7
        in = 1'b0; #10;  // A6
        in = 1'b0; #10;  // A5
        in = 1'b0; #10;  // A4
        in = 1'b1; #10;  // A3
        in = 1'b1; #10;  // A2
        in = 1'b0; #10;  // A1
        in = 1'b1; #10;  // A0
```



```
tsenyuch@eng-analogsim3:~/ECE499/cadence/t
 GNU nano 5.6.1                                                  tb_ser
        // Send B = 00001001 (9), MSB first
        in = 1'b0; #10;  // B7
        in = 1'b0; #10;  // B6
        in = 1'b0; #10;  // B5
        in = 1'b0; #10;  // B4
        in = 1'b1; #10;  // B3
        in = 1'b0; #10;  // B2
        in = 1'b0; #10;  // B1
        in = 1'b1; #10;  // B0
    end

    // Capture serial output when SHIFT_OUT begins
    initial begin
        out_capture = 16'b0;

        wait(reset == 1'b0);
        wait(start == 1'b1);

        // Wait until the FSM enters the output shifting phase
        wait(uut.p2s_shift_en == 1'b1);

        // Small delay to allow out to settle
        #1;

        $display("Expected output = 0000000001110101");
        $write("Captured output = ");

        // Capture first output bit immediately
        out_capture[15] = out;
        $write("%b", out);

        // Capture remaining 15 bits on subsequent clock cycles
        for (i = 1; i < 16; i = i + 1) begin
            @(posedge clk);
            #1;
            out_capture[15-i] = out;
            $write("%b", out);
        end

        $display("");

        #1;
        if (out_capture == 16'b0000000001110101)
            $display("PASS: serial_multiplier8x8_top works correctly.");
        else begin
            $display("FAIL: output mismatch.");
            $display("Captured = %b", out_capture);
            $display("Expected = 0000000001110101");
        end

        wait(done == 1'b1);
        $display("done asserted at time %0t", $time);

        #10;
```

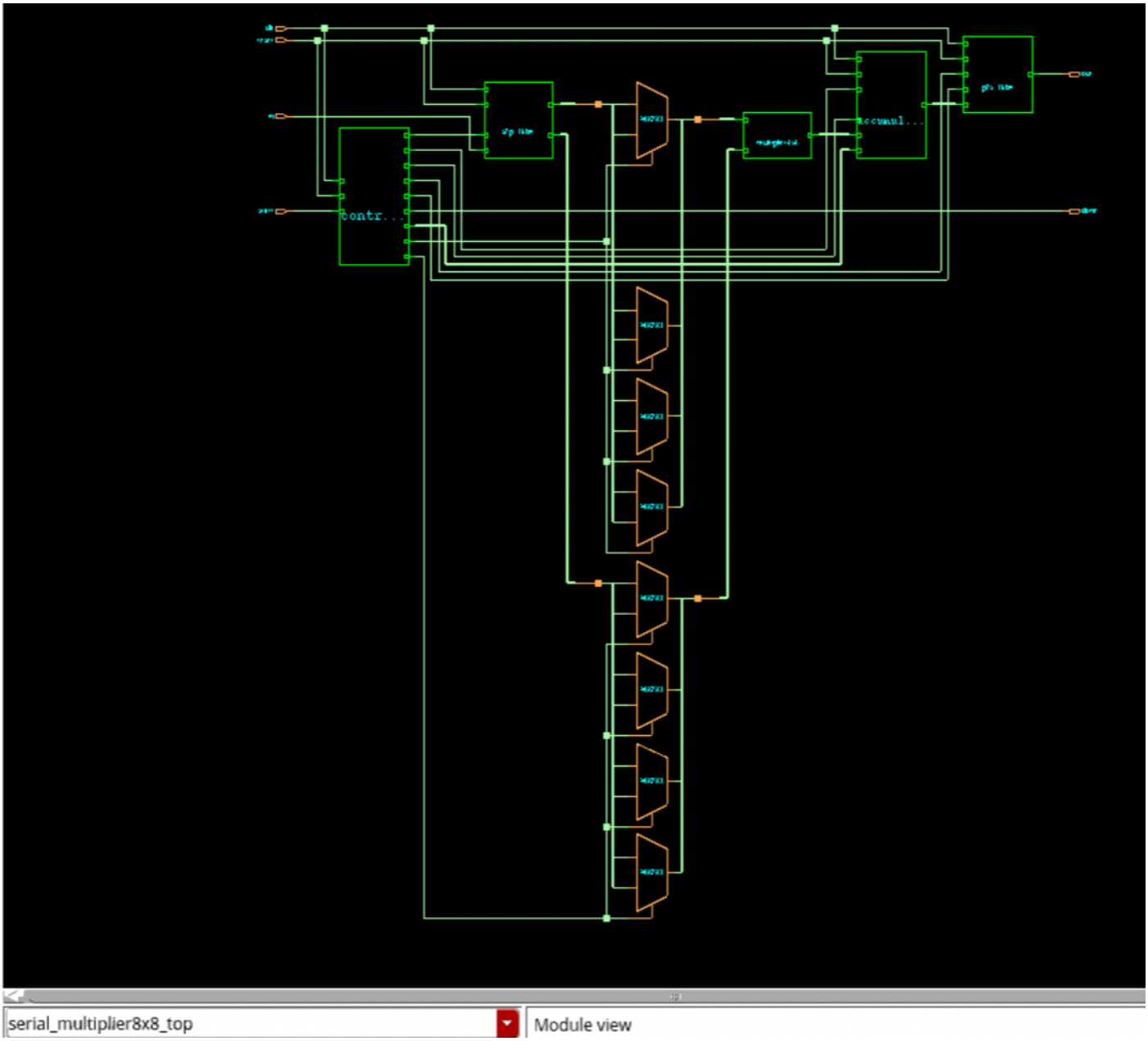
The testbench verifies the serial multiplier by applying serial input data and capturing the output bit-by-bit. Instead of using a fixed delay, it waits for the `p2s_shift_en` signal to ensure proper timing alignment before capturing output data. The captured result is compared with the expected value, and a PASS/FAIL message is generated automatically.

4.4 Simulation Results



The waveform shows the complete operation of the serial multiplier, including input loading, computation, and output shifting. The operands are serially loaded via the `in` signal, and the result is shifted out through `out`. The captured output progresses to the final value 0x0075, which matches the expected result of 13 × 9. The done signal indicates completion of the operation.

4.5 Synthesis Results



The synthesized 8×8 serial multiplier meets the timing requirement under a 10 ns clock constraint with a positive slack of 0.049 ns, indicating that the design satisfies timing with a near-critical path delay. Based on this slack, the maximum path delay is approximately 9.951 ns, which corresponds to a maximum operating frequency of about 100.5 MHz.

The total cell area of the design is 10,938.7 μm^2. Among all modules, the accumulate unit contributes the largest portion of the area, followed by the FSM controller and the parallel-to-serial (P2S) block.

The total power consumption is approximately 0.384 mW. The majority of the power is dominated by internal (62.87%) and switching (36.95%) components, while leakage power is negligible.

The execution time of the multiplier is determined by the FSM operation. The design requires 16 cycles to load inputs, 1 cycle to clear the accumulator, 4 cycles for partial-product computation, 1 cycle to load the output register, and 16 cycles to shift out the result, resulting in a total of 38 clock cycles. Therefore, the total latency is approximately 380 ns under a 10 ns clock period. If the DONE state is included, the total latency becomes 390 ns.

5. Discussion

• How does each block work? Describe operation in terms of inputs, output, and states (where appropriate).

The design is composed of several main functional blocks: a serial-to-parallel (S2P) input register, a finite state machine (FSM) controller, a multiplier datapath, an accumulator, and a parallel-to-serial (P2S) output register.

The S2P block receives serial input bits and converts them into parallel operands. The FSM controller governs the overall operation of the system, including loading inputs, clearing registers, enabling multiplication steps, and shifting out results. The multiplier datapath generates partial products based on the input operands, while the accumulator stores and updates intermediate results across multiple cycles. Finally, the P2S block converts the parallel multiplication result into a serial output stream.

The FSM transitions through multiple states including input loading, computation, and output shifting, ensuring correct sequencing of operations and synchronization between all blocks.

• How did you test your design and ensure that it functions correctly with different operands and operations?

The design was verified using a SystemVerilog testbench in Xcelium. Multiple test cases were applied to ensure correctness, including different combinations of input operands such as small values, large values, and edge cases (e.g., 0, maximum values).

Waveform analysis was performed using SimVision to confirm correct timing behavior and functional correctness. Key signals such as input loading, state transitions, accumulator updates, and output shifting were carefully examined. The output results were compared against expected multiplication values to ensure accuracy.

Additionally, corner cases such as reset behavior and sequential input loading were verified to confirm robust operation of the design.

• What were the primary challenges that you encountered while working on the project?

One of the primary challenges in this project was coordinating the timing between different modules, especially ensuring correct synchronization between the FSM controller and datapath operations. Managing multi-cycle operations, such as serial input loading and output shifting, required careful design of state transitions and counters.

Another challenge was debugging incorrect intermediate results due to timing or control signal issues. This required extensive waveform analysis to trace signal propagation and identify logic errors.

Additionally, understanding the synthesis results and interpreting timing and power reports from Genus required careful analysis to ensure that the design met performance constraints.

• Is there anything you would implement differently if you were to redesign this project?

If the design were to be improved, one potential enhancement would be to increase performance by adopting a parallel multiplier architecture instead of a serial implementation. This would significantly reduce the number of clock cycles required for computation at the cost of increased area.

Another improvement would be optimizing the FSM and datapath to reduce unnecessary states or transitions, thereby improving efficiency. Power optimization techniques, such as clock gating, could also be applied to reduce switching activity.

Finally, further optimization of the synthesis constraints and exploration of different effort levels could improve timing and area trade-offs.